

Омск 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. Описание предметной области	5
2. Концептуальное проектирование БД	7
3. Логическое проектирование БД	9
4. Физическая модель БД	12
5. Реализация БД в СУБД PostgreSQL	15
5.1 Создание таблиц	15
5.2. Настройка ограничений данных	16
5.2.1. Проверочные ограничения (CHECK)	17
5.2.2. Ограничения уникальности (UNIQUE)	18
5.2.3. Внешние ключи (FOREIGN KEY)	19
5.3. Создание индексов	21
5.4. Заполнение таблиц данными	22
6. Разработка запросов к БД	23
6.1. Запрос на выборку данных	23
6.1.1. Однотабличный запрос с простым условием	23
6.1.2. Однотабличные запросы со сложным условием	23
6.1.3. Многотабличный запрос с вычисляемым полем и простым условием	25
6.1.4. Многотабличные запросы со сложными условиями	26
6.2. Запросы с использованием оконных функций	30
6.3. Запросы с вычисляемыми полями	32
6.4. Запросы с агрегатными функциями	34
6.5. Запросы с использованием подзапросов	36
6.6. Запрос на основе материализованного представления	39
7. Разработка объектов БД	40
7.1. Разработка материализованных представлений	40
7.2. Разработка курсоров	42
7.3. Разработка функции	43
7.4. Разработка процедуры	44

7.5. Разработка триггеров	46
ЗАКЛЮЧЕНИЕ	49
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	50

ВВЕДЕНИЕ

В современных условиях хозяйственной деятельности предприятий, предоставляющих коммунальные услуги, возрастает необходимость в эффективном управлении процессами учёта, начисления и оплаты услуг. Ручная обработка данных, связанных с потреблением воды, электроэнергии и газа, а также расчётом платежей, является трудоёмкой, подвержена ошибкам и не обеспечивает необходимой оперативности. Внедрение автоматизированных систем на основе баз данных позволяет оптимизировать эти процессы, повысить точность расчётов, обеспечить контроль за платежами и улучшить взаимодействие с потребителями.

Данная работа посвящена проектированию и реализации базы данных для автоматизации учёта оплаты коммунальных услуг, предоставляемых физическим и юридическим лицам. Цель работы — создать структурированное, нормализованное хранилище данных, которое позволит вести учёт клиентов, договоров, приборов учёта, фиксировать показания, рассчитывать начисления на основе актуальных тарифов, регистрировать платежи и автоматически распределять их по неоплаченным счетам.

В процессе работы были решены следующие задачи:

- Проведён анализ предметной области, выделены ключевые сущности и бизнес-правила.
- Разработана концептуальная и логическая модель данных в нотации IDEF1X.
- Реализована физическая модель в СУБД PostgreSQL с созданием таблиц, ограничений целостности и индексов.
- Разработан комплекс запросов различной сложности для анализа данных и формирования отчётности.
- Созданы объекты базы данных: материализованные представления, курсоры, функции, процедуры и триггеры для автоматизации бизнес-процессов.

1. Описание предметной области

Предприятие-поставщик предоставляет населению и организациям коммунальные услуги: воду, электричество и газ. Для каждого клиента ведётся лицевой счёт, к которому привязаны приборы учёта (счётчики). Ежемесячно снимаются показания, на основе которых автоматически рассчитывается сумма к оплате по действующим тарифам. Клиенты вносят платежи, которые система автоматически распределяет по неоплаченным счетам.

Основные задачи базы данных:

- Учёт клиентов (физических и юридических лиц) и их контактных данных.
- Регистрация договоров на предоставление услуг, связывающих клиентов с конкретными видами услуг.
- Учёт приборов учёта (счётчиков), их типов, моделей и мест установки.
- Фиксация периодических показаний счётчиков с контролем корректности (текущие > предыдущие).
- Хранение актуальных и исторических тарифов на каждый вид услуги.
- Автоматический расчёт сумм начисления по формуле:
(Текущие показания – Предыдущие показания) × Тариф.
- Учёт поступающих от клиентов платежей.
- Автоматическое распределение платежей по неоплаченным начислениям (принцип FIFO или по указанию клиента).
- Расчёт текущего баланса по лицевому счёту (задолженность/переплата).
- Контроль соблюдения бизнес-правил и обеспечение целостности данных.

В процессе разработки были выделены следующие сущности: клиент, услуга, договор, счётчик, показание счётчика, тариф, начисление, платёж, распределение платежа

В предметной области действуют следующие бизнес-правила:

- Клиентом может быть физическое лицо (ФИО, паспорт) или юридическое лицо (название, ИНН).
- У абонента может быть несколько договоров на разные услуги.
- Каждый договор связан с конкретной услугой (вода, свет, газ).
- По каждому договору регулярно снимаются показания счётчиков (если установлены).
- На основе показаний и тарифов рассчитывается сумма к оплате по следующей формуле:
$$(\text{Текущие показания} - \text{Предыдущие показания}) \times \text{Тариф}.$$
- Фиксируются все платежи и начисления.
- При наличии задолженности начисляется пеня.
- Тарифы на услуги могут меняться. При расчёте используется тариф, действующий на дату потребления.
- При наличии задолженности начисляется пеня.
- Расчёты должны вестись с точностью до 2 знаков после запятой (для денежных сумм).

2. Концептуальное проектирование БД

Концептуальная модель отражает ключевые сущности и связи между ними.

Выделенные сущности:

1. Клиент (Client) — физические и юридические лица, получающие услуги.
 - Идентификатор клиента
 - Тип клиента (физическое/юридическое лицо)
 - ФИО / Название организации
 - Паспортные данные / ИНН
 - Телефон
 - Электронная почта
 - Адрес
2. Услуга (Service) — тип предоставляемой коммунальной услуги (водоснабжение, электроснабжение, газоснабжение).
 - Идентификатор услуги
 - Название услуги (вода, электричество, газ)
 - Единица измерения (м³, кВт·ч)
3. Договор (Contract) — документ, связывающий клиента с конкретной услугой.
 - Идентификатор договора
 - Номер договора
 - Дата заключения
 - Дата окончания
 - Ссылка на клиента
 - Ссылка на услугу
 - Статус договора(активен, расторгнут)
4. Счётчик (Meter) — прибор учёта, привязанный к договору.

- Идентификатор счётчика
 - Серийный номер
 - Модель
 - Дата установки
 - Место установки
 - Ссылка на договор
5. Показание (MeterReading) — ежемесячное показание конкретного счётчика.
- Идентификатор показания
 - Ссылка на счётчик
 - Дата снятия показания
 - Текущее показание
 - Предыдущее показание
 - Расход (вычисляемое поле: текущее – предыдущее)
6. Тариф (Tariff) — цена за единицу услуги, действующая в определённый период.
- Идентификатор тарифа
 - Ссылка на услугу
 - Дата начала действия
 - Дата окончания действия
 - Стоимость за единицу
7. Начисление (Assgual) — рассчитанная к оплате сумма за период по договору.
- Идентификатор начисления
 - Ссылка на договор
 - Период начисления (месяц, год)
 - Расход по услуге
 - Сумма к оплате (вычисляемое поле: расход × тариф)
 - Дата формирования

- Статус (оплачено/не оплачено)
8. Платёж (Payment) — факт внесения денежных средств клиентом.
- Идентификатор платежа
 - Ссылка на клиента
 - Дата платежа
 - Сумма платежа
 - Способ оплаты (карта, наличные, перевод)
9. Распределение платежа (PaymentAllocation) — связь платежа с погашаемыми начислениями (для реализации FIFO или иного алгоритма).
- Идентификатор распределения
 - Ссылка на платеж
 - Ссылка на начисление
 - Сумма распределения

3. Логическое проектирование БД

Описание связей между сущностями:

- Клиент — Договор: 1:M. Один клиент может иметь несколько договоров.
- Услуга — Договор: 1:M. На одну услугу может быть заключено множество договоров.
- Договор — Счётчик: 1:M. По одному договору может быть установлено несколько счётчиков (например, отдельно на горячую и холодную воду).
- Счётчик — Показание: 1:M. С каждого счётчика снимаются показания ежемесячно.
- Услуга — Тариф: 1:M. Для каждой услуги тарифы меняются с течением времени.

- Договор — Начисление: 1:M. По каждому договору формируются ежемесячные начисления.
- Клиент — Платёж: 1:M. Один клиент может совершать множество платежей.
- Начисление — Распределение платежа: 1:M. Одно начисление может быть оплачено частями (связующая сущность).
- Платёж — Распределение платежа: 1:M. Один платёж может распределяться на несколько начислений (связующая сущность).

Логическая модель в нотации IDEF1X представлена на рисунке 1.

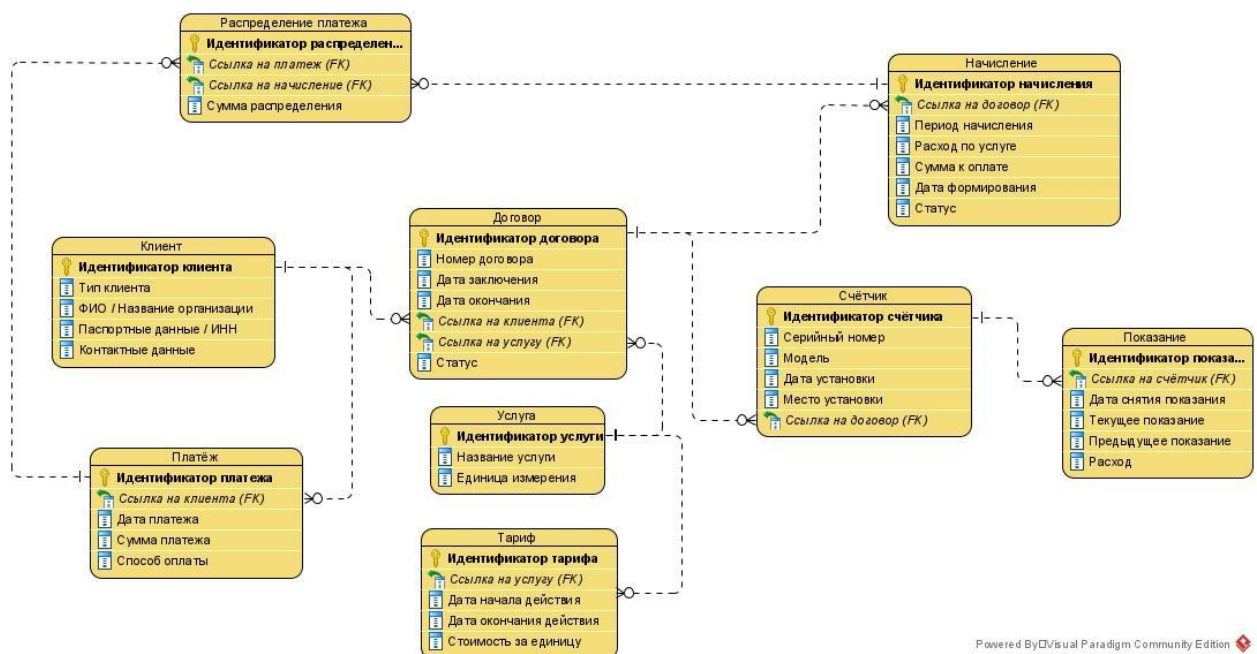


Рисунок 1. — Логическая модель данных

Нормализация:

Все таблицы находятся в третьей нормальной форме (3НФ):

- 1НФ: Все атрибуты атомарны, нет повторяющихся групп.
- 2НФ: Все неключевые атрибуты полностью зависят от первичного ключа.
- 3НФ: Отсутствуют транзитивные зависимости. Все справочные данные (услуги, тарифы) вынесены в отдельные таблицы.

4. Физическая модель БД

Таблицы и их структура:

1. *clients*

- *client_id* SERIAL PRIMARY KEY
- *client_type* VARCHAR(20) NOT NULL
- *full_name* VARCHAR(255) NOT NULL
- *identity_doc* VARCHAR(100) NOT NULL
- *phone* VARCHAR(20) NOT NULL
- *email* VARCHAR(100)
- *address* TEXT

2. *services*

- *service_id* SERIAL PRIMARY KEY
- *service_name* VARCHAR(100) NOT NULL
- *unit* VARCHAR(20) NOT NULL

3. *contracts*

- *contract_id* SERIAL PRIMARY KEY
- *contract_number* VARCHAR(50) NOT NULL
- *client_id* INTEGER NOT NULL REFERENCES *clients*(*client_id*) ON

DELETE CASCADE

- *service_id* INTEGER NOT NULL REFERENCES *services*(*service_id*)

ON DELETE RESTRICT

- *start_date* DATE NOT NULL
- *end_date* DATE
- *status* VARCHAR(20) NOT NULL DEFAULT 'Активен'

4. *meters*

- *meter_id* SERIAL PRIMARY KEY
- *serial_number* VARCHAR(100) NOT NULL
- *model* VARCHAR(100) NOT NULL

- *installation_date* DATE NOT NULL
- *location* TEXT NOT NULL
- *contract_id* INTEGER NOT NULL REFERENCES contracts(*contract_id*) ON DELETE CASCADE

5. *meter_readings*

- *reading_id* SERIAL PRIMARY KEY
- *meter_id* INTEGER NOT NULL REFERENCES meters(*meter_id*) ON DELETE CASCADE

- *reading_date* DATE NOT NULL
- *current_value* DECIMAL(15, 3) NOT NULL
- *previous_value* DECIMAL(15, 3) NOT NULL
- *consumption* DECIMAL(15, 3) GENERATED ALWAYS AS (*current_value* - *previous_value*) STORED

6. *tariffs*

- *tariff_id* SERIAL PRIMARY KEY
- *service_id* INTEGER NOT NULL REFERENCES services(*service_id*) ON DELETE CASCADE

- *start_date* DATE NOT NULL
- *end_date* DATE NOT NULL
- *rate* DECIMAL(10, 2) NOT NULL

7. *accruals*

- *accrual_id* SERIAL PRIMARY KEY
- *contract_id* INTEGER NOT NULL REFERENCES contracts(*contract_id*) ON DELETE CASCADE

- *period* DATE NOT NULL -- Первое число месяца начисления
- *consumption* DECIMAL(15, 3) NOT NULL
- *amount* DECIMAL(15, 2) NOT NULL
- *created_date* DATE NOT NULL DEFAULT CURRENT_DATE
- *status* VARCHAR(20) NOT NULL DEFAULT 'Не оплачено'

8. payments

- *payment_id* SERIAL PRIMARY KEY
- *client_id* INTEGER NOT NULL REFERENCES clients(*client_id*) ON

DELETE CASCADE

- *payment_date* DATE NOT NULL DEFAULT CURRENT_DATE
- *amount* DECIMAL(15, 2) NOT NULL
- *payment_method* VARCHAR(50) NOT NULL

9. payment_allocations

- *allocation_id* SERIAL PRIMARY KEY
- *payment_id* INTEGER NOT NULL REFERENCES

payments(*payment_id*) ON DELETE CASCADE

- *accrual_id* INTEGER NOT NULL REFERENCES

accruals(*accrual_id*) ON DELETE CASCADE

- *allocated_amount* DECIMAL(15, 2) NOT NULL

Физическая модель в нотации IDEF1X представлена на рисунке 2.

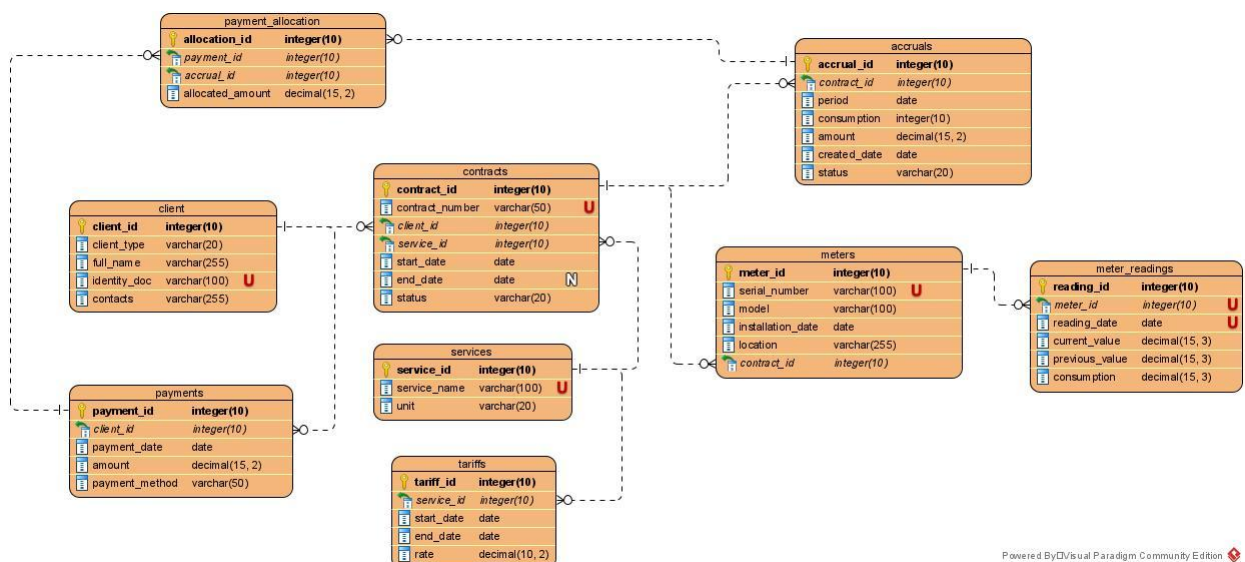


Рисунок 2. – Физическая модель данных

5. Реализация БД в СУБД PostgreSQL

5.1 Создание таблиц

Развёртывание базы данных для автоматизированной системы расчёта коммунальных платежей выполнялось в PostgreSQL с применением команд DDL. На основании разработанной физической модели были созданы девять взаимосвязанных таблиц, отражающих ключевые бизнес-сущности системы учёта услуг.

Состав основных объектов базы:

1. clients — содержит сведения о потребителях услуг;
2. services — перечень предоставляемых ресурсов (вода, электричество, газ);
3. contracts — регистр договоров на обслуживание;
4. meters — учёт приборов измерения потребления;
5. meter_readings — фиксация показаний по периодам;
6. tariffs — архив действующих и исторических расценок;
7. accruals — начисления за фактическое потребление;
8. payments — операции по внесению средств;
9. payment_allocations — связь платежей и погашаемых начислений.

Для идентификации записей всех таблиц используются автоинкрементные целочисленные ключи (SERIAL);

Связи между таблицами реализованы через внешние ключи с политиками CASCADE и RESTRICT, что поддерживает целостность при изменениях;

Для хранения финансовых величин и объёмов потребления выбран тип DECIMAL с фиксированной точностью, что исключает ошибки округления в расчётах;

Временные параметры (даты заключения договоров, периоды начислений) используют тип DATE;

Вычисляемое поле в таблице показаний автоматически определяет расход на основе разницы текущего и предыдущего значений.

Графическое представление созданной схемы в виде диаграммы сущностей-связей приведено на рисунке 3.

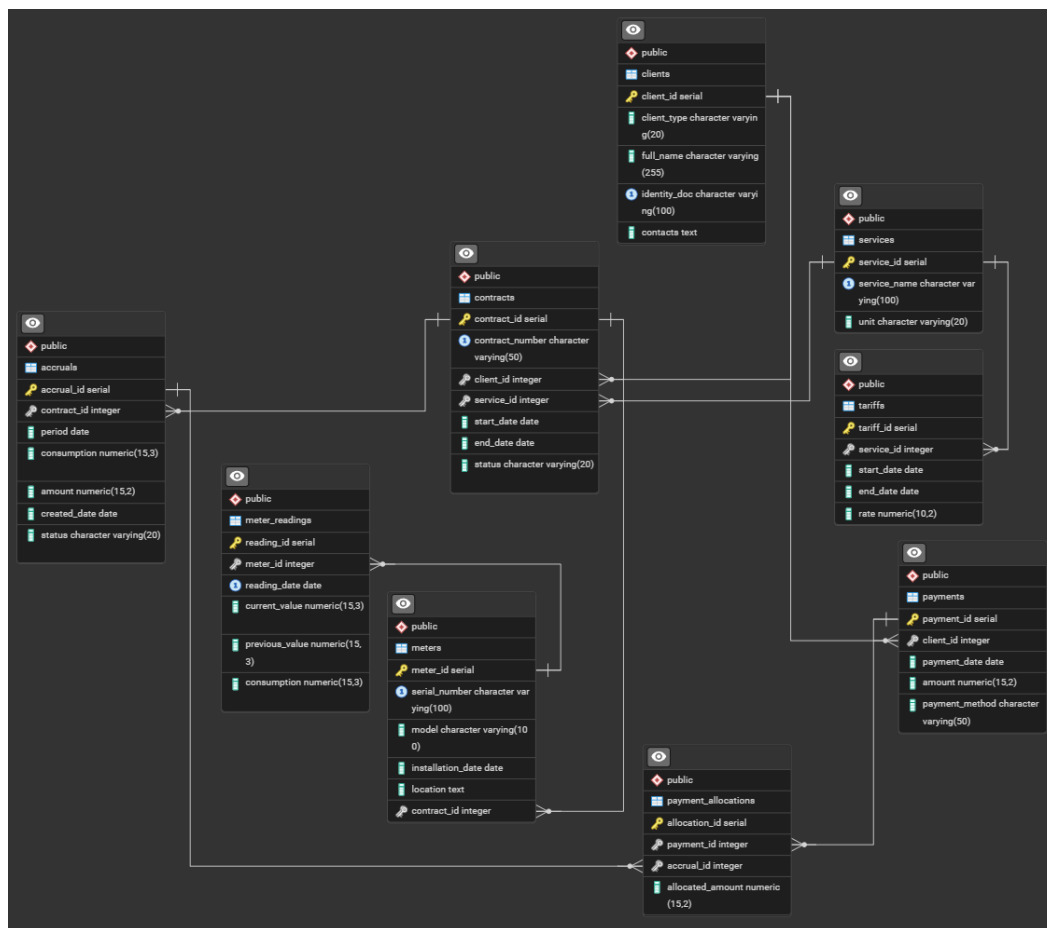


Рисунок 3. – Схема базы данных в нотации «сущность–связь»

5.2. Настройка ограничений данных

После формирования каркаса таблиц выполнена детальная настройка проверочных условий, обеспечивающих семантическую корректность хранимой информации. Ограничения добавлены отдельными командами ALTER TABLE, что соответствует принятой практике разделения создания структуры и наложения бизнес-правил.

SQL скрипты для создания ограничений представлены на рисунке 4.


```

1 ALTER TABLE clients
2 ADD CONSTRAINT chk_client_type
3 CHECK (client_type IN ('Физическое лицо', 'Юридическое лицо'));
4
5 ALTER TABLE clients
6 ADD CONSTRAINT uniq_identity_doc UNIQUE (identity_doc);
7
8 ALTER TABLE services
9 ADD CONSTRAINT uniq_service_name UNIQUE (service_name);
10
11 ALTER TABLE contracts
12 ADD CONSTRAINT chk_contract_status
13 CHECK (status IN ('Активен', 'Расторгнут'));
14
15 ALTER TABLE contracts
16 ADD CONSTRAINT uniq_contract_number UNIQUE (contract_number);
17
18 ALTER TABLE meters
19 ADD CONSTRAINT uniq_serial_number UNIQUE (serial_number);
20
21 ALTER TABLE meter_readings
22 ADD CONSTRAINT chk_current_value_positive
23 CHECK (current_value >= 0);
24
25 ALTER TABLE meter_readings
26 ADD CONSTRAINT chk_previous_value_positive
27 CHECK (previous_value >= 0);
28
29 ALTER TABLE meter_readings
30 ADD CONSTRAINT chk_current_gte_previous
31 CHECK (current_value >= previous_value);
32
33 ALTER TABLE meter_readings
34 ADD CONSTRAINT uniq_meter_reading_date UNIQUE (meter_id, reading_date);
35
36 ALTER TABLE tariffs
37 ADD CONSTRAINT chk_rate_positive
38 CHECK (rate > 0);
39
40 ALTER TABLE tariffs
41 ADD CONSTRAINT chk_dates_valid
42 CHECK (end_date > start_date);
43
44 ALTER TABLE accruals
45 ADD CONSTRAINT chk_consumption_positive
46 CHECK (consumption >= 0);
47
48 ALTER TABLE accruals
49 ADD CONSTRAINT chk_amount_positive
50 CHECK (amount >= 0);
51
52 ALTER TABLE accruals
53 ADD CONSTRAINT chk_accrual_status
54 CHECK (status IN ('Не оплачено', 'Частично оплачено', 'Оплачено'));
55
56 ALTER TABLE payments
57 ADD CONSTRAINT chk_payment_amount_positive
58 CHECK (amount > 0);
59
60 ALTER TABLE payments
61 ADD CONSTRAINT chk_payment_method
62 CHECK (payment_method IN ('Карта', 'Наличные', 'Банковский перевод'));
63
64 ALTER TABLE payment_allocations
65 ADD CONSTRAINT chk_allocated_amount_positive
66 CHECK (allocated_amount > 0);

```

Рисунок 4. – Ограничения данных

5.2.1. Проверочные ограничения (CHECK)

Для каждой таблицы определены специфические проверки, соответствующие требованиям предметной области:

Таблица clients (клиенты):

- Ограничение `chk_client_type` гарантирует, что поле `client_type` содержит только допустимые значения: «Физическое лицо» или «Юридическое лицо». Это обеспечивает корректную классификацию потребителей услуг.

Таблица contracts (договоры):

- Ограничение `chk_contract_status` проверяет, что статус договора соответствует одному из разрешённых состояний: «Активен» или «Расторгнут». Это поддерживает корректность бизнес-процессов управления договорными отношениями.

Таблица meter_readings (показания счётчиков):

- Ограничение `chk_current_value_positive` и `chk_previous_value_positive` обеспечивают неотрицательность показаний счётчиков, что соответствует физическому смыслу измерений.

- Ограничение `chk_current_gte_previous` гарантирует, что текущее показание не меньше предыдущего, предотвращая ошибочный ввод данных, где расход был бы отрицательным.

Таблица `tariffs` (тарифы):

- Ограничение `chk_rate_positive` требует, чтобы стоимость тарифа была строго больше нуля, исключая нулевые или отрицательные расценки.
- Ограничение `chk_dates_valid` проверяет корректность временного интервала действия тарифа: дата окончания должна быть позже даты начала.

Таблица `accruals` (начисления):

- Ограничения `chk_consumption_positive` и `chk_amount_positive` обеспечивают неотрицательность значений расхода и суммы начисления.
- Ограничение `chk_accrual_status` контролирует, что статус начисления соответствует одному из трёх допустимых состояний: «Не оплачено», «Частично оплачено» или «Оплачено».

Таблица `payments` (платежи):

- Ограничение `chk_payment_amount_positive` требует, чтобы сумма платежа была больше нуля.
- Ограничение `chk_payment_method` проверяет, что способ оплаты соответствует одному из разрешённых вариантов: «Карта», «Наличные» или «Банковский перевод».

Таблица `payment_allocations` (распределения платежей):

- Ограничение `chk_allocated_amount_positive` гарантирует, что распределяемая сумма строго положительна.

5.2.2. Ограничения уникальности (UNIQUE)

Для обеспечения уникальности ключевых бизнес-идентификаторов установлены следующие ограничения:

Таблица `clients`:

- Ограничение `uniq_identity_doc` гарантирует уникальность документа, удостоверяющего личность (паспортные данные или ИНН), что предотвращает дублирование записей о клиентах.

Таблица `services`:

- Ограничение `uniq_service_name` обеспечивает уникальность названий услуг (вода, электричество, газ) в справочнике.

Таблица `contracts`:

- Ограничение `uniq_contract_number` требует уникальности номера договора в системе, что исключает конфликты при идентификации договорных отношений.

Таблица `meters`:

- Ограничение `uniq_serial_number` гарантирует уникальность серийного номера прибора учёта, что необходимо для однозначной идентификации счётчиков.

Таблица `meter_readings`:

- Составное ограничение `uniq_meter_reading_date` предотвращает дублирование показаний для одного счётчика за один период, обеспечивая уникальность комбинации «идентификатор счётчика + дата показания».

5.2.3. Внешние ключи (FOREIGN KEY)

Связи между таблицами, определённые при создании структуры, обеспечивают ссылочную целостность:

В таблице `contracts` внешние ключи связывают договоры с клиентами (`client_id`) и услугами (`service_id`), с политиками `ON DELETE CASCADE` и `ON DELETE RESTRICT` соответственно.

- В таблице `meters` внешний ключ `contract_id` обеспечивает привязку счётчиков к договорам с каскадным удалением.

- В таблице `meter_readings` внешний ключ `meter_id` связывает показания с конкретными приборами учёта.

- В таблице tariffs внешний ключ service_id привязывает тарифы к услугам.
- В таблице accruals внешний ключ contract_id связывает начисления с договорами.
- В таблице payments внешний ключ client_id обеспечивает связь платежей с клиентами.
- В таблице payment_allocations внешние ключи связывают распределения с платежами и начислениями.

5.3. Создание индексов

Для оптимизации производительности запросов и ускорения операций поиска и соединения в базе данных созданы дополнительные индексы помимо автоматически формируемых для первичных ключей. Индексы разработаны с учётом типичных сценариев использования системы учёта коммунальных услуг.

Все индексы созданы по типу B-tree, который оптимален для диапазонных запросов и операций сравнения. Реализация индексов показана на рисунках 5 и 6.

```
CREATE INDEX idx_contracts_client_id ON contracts(client_id);
CREATE INDEX idx_contracts_service_id ON contracts(service_id);
CREATE INDEX idx_meters_contract_id ON meters(contract_id);
CREATE INDEX idx_meter_readings_meter_id ON meter_readings(meter_id);
CREATE INDEX idx_tariffs_service_id ON tariffs(service_id);
CREATE INDEX idx_accruals_contract_id ON accruals(contract_id);
CREATE INDEX idx_payments_client_id ON payments(client_id);
CREATE INDEX idx_payment_allocations_payment_id ON payment_allocations(payment_id);
CREATE INDEX idx_payment_allocations_accrual_id ON payment_allocations(accrual_id);
```

Рисунок 5. – Индексы для внешних ключей (ускорение JOIN-операций)

```
-- Для быстрого поиска клиентов по документу
CREATE INDEX idx_clients_identity_doc ON clients(identity_doc);

-- Для поиска договоров по номеру
CREATE INDEX idx_contracts_contract_number ON contracts(contract_number);

-- Для поиска счётчиков по серийному номеру
CREATE INDEX idx_meters_serial_number ON meters(serial_number);

-- Для временных диапазонов в показаниях и начислениях
CREATE INDEX idx_meter_readings_reading_date ON meter_readings(reading_date);
CREATE INDEX idx_accruals_period ON accruals(period);
CREATE INDEX idx_accruals_status ON accruals(status);
CREATE INDEX idx_payments_payment_date ON payments(payment_date);

-- Для поиска тарифов по периоду действия
CREATE INDEX idx_tariffs_dates ON tariffs(start_date, end_date);
```

Рисунок 6. – Индексы для часто используемых полей фильтрации

5.4. Заполнение таблиц данными

Для тестирования функциональности и демонстрации работы системы таблицы заполнены тестовыми данными, отражающими реальные сценарии использования.

Результат заполнения таблицы *clients* показан на рисунке 7.

	client_id [PK] integer	client_type character varying (20)	full_name character varying (255)	identity_doc character varying (100)	contacts text
1	1	Физическое лицо	Иванов Иван Иванович	4505 123456	+7-900-123-45-67, ivanov@mail.ru
2	2	Физическое лицо	Петрова Анна Сергеевна	4510 654321	+7-901-234-56-78, petrova@yandex...
3	3	Юридическое лицо	ООО "Вектор"	7712345678	+7-495-123-45-67, director@vector...
4	4	Физическое лицо	Сидоров Алексей Петрович	4515 789012	+7-902-345-67-89
5	5	Юридическое лицо	ИП Козлова М.В.	7723456789	+7-916-456-78-90, kozlova@mail.ru
6	6	Физическое лицо	Николаева Елена Викторовна	4520 345678	+7-903-456-78-91
7	7	Физическое лицо	Фёдоров Дмитрий Алексеевич	4525 901234	+7-904-567-89-12
8	8	Физическое лицо	Волкова Ольга Игоревна	4530 567890	+7-905-678-90-13
9	9	Юридическое лицо	ООО "СтройГарант"	7734567890	+7-495-234-56-78, info@stroigarant...
10	10	Физическое лицо	Киселёв Павел Викторович	4535 123789	+7-906-789-01-24
11	11	Физическое лицо	Морозова Татьяна Сергеевна	4540 456123	+7-907-890-12-35
12	12	Юридическое лицо	ЗАО "ТехноПрофи"	7745678901	+7-495-345-67-89
13	13	Физическое лицо	Белов Андрей Николаевич	4545 789456	+7-908-901-23-46
14	14	Физическое лицо	Григорьева Ирина Петровна	4550 123654	+7-909-012-34-57
15	15	Юридическое лицо	ИП Семёнов Р.К.	7756789012	+7-910-123-45-68
16	16	Физическое лицо	Данилов Михаил Олегович	4555 456987	+7-911-234-56-79
17	17	Физическое лицо	Орлова Светлана Владимиров...	4560 789321	+7-912-345-67-80
18	18	Физическое лицо	Титов Виктор Александрович	4565 321654	+7-913-456-78-91

Рисунок 7. – Таблица *clients* с тестовыми данными

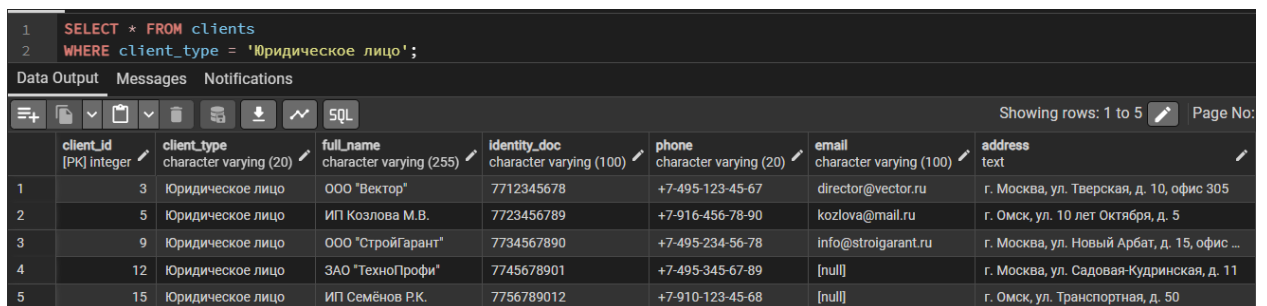
6. Разработка запросов к БД

6.1. Запрос на выборку данных

6.1.1. Однотабличный запрос с простым условием

Задача: Получить список всех клиентов, которые являются юридическими лицами, для подготовки рассылки официальных документов.

Результат выполнения запроса представлен на рисунке 8.



Showing rows: 1 to 5 Page No:							
	client_id [PK] integer	client_type character varying (20)	full_name character varying (255)	identity_doc character varying (100)	phone character varying (20)	email character varying (100)	address text
1	3	Юридическое лицо	ООО "Вектор"	7712345678	+7-495-123-45-67	director@vector.ru	г. Москва, ул. Тверская, д. 10, офис 305
2	5	Юридическое лицо	ИП Козлова М.В.	7723456789	+7-916-456-78-90	kozlova@mail.ru	г. Омск, ул. 10 лет Октября, д. 5
3	9	Юридическое лицо	ООО "СтройГарант"	7734567890	+7-495-234-56-78	info@stroigarant.ru	г. Москва, ул. Новый Арбат, д. 15, офис ...
4	12	Юридическое лицо	ЗАО "ТехноПрофи"	7745678901	+7-495-345-67-89	[null]	г. Москва, ул. Садовая-Кудринская, д. 11
5	15	Юридическое лицо	ИП Семёнов Р.К.	7756789012	+7-910-123-45-68	[null]	г. Омск, ул. Транспортная, д. 50

Рисунок 8. – Результат выполнения запроса на выборку юридических лиц

6.1.2. Однотабличные запросы со сложным условием

б1. Запрос с условиями AND, AND, NOT:

Задача: Найти все активные договоры, заключённые в 2023 году, но не касающиеся услуги газоснабжения, для анализа загрузки отделов по другим видам услуг.

Результат выполнения запроса представлен на рисунке 9.

1	SELECT *
2	FROM contracts
3	WHERE EXTRACT(YEAR FROM start_date) = 2023
4	AND status = 'Активен'
5	AND service_id != 3;

	contract_id [PK] integer	contract_number character varying (50)	client_id integer	service_id integer	start_date date	end_date date	status character varying (20)
1	1	ВОД-2023-001	1	1	2023-01-15	[null]	Активен
2	2	ЭЛ-2023-001	1	2	2023-02-10	[null]	Активен
3	4	ВОД-2023-002	3	1	2023-01-20	2024-12-31	Активен
4	5	ЭЛ-2023-002	3	2	2023-02-15	[null]	Активен
5	6	ВОД-2023-003	4	1	2023-04-01	[null]	Активен
6	8	ЭЛ-2023-003	6	2	2023-06-01	[null]	Активен
7	9	ВОД-2023-004	7	1	2023-07-05	[null]	Активен
8	10	ЭЛ-2023-004	8	2	2023-08-12	[null]	Активен
9	12	ВОД-2023-005	10	1	2023-10-20	[null]	Активен
10	13	ЭЛ-2023-005	11	2	2023-11-01	2024-10-31	Активен

Рисунок 9. – Активные договоры 2023 года (кроме газоснабжения)

b2. Запрос с условиями AND, OR:

Задача: Найти показания счётчиков, снятые после 1 февраля 2024 года, у которых либо текущее значение аномально высокое (>100), либо зафиксирован большой расход (>10).

Результат выполнения запроса представлен на рисунке 10.

1	SELECT *
2	FROM meter_readings
3	WHERE reading_date > '2024-02-01'
4	AND (current_value > 100 OR consumption > 10);

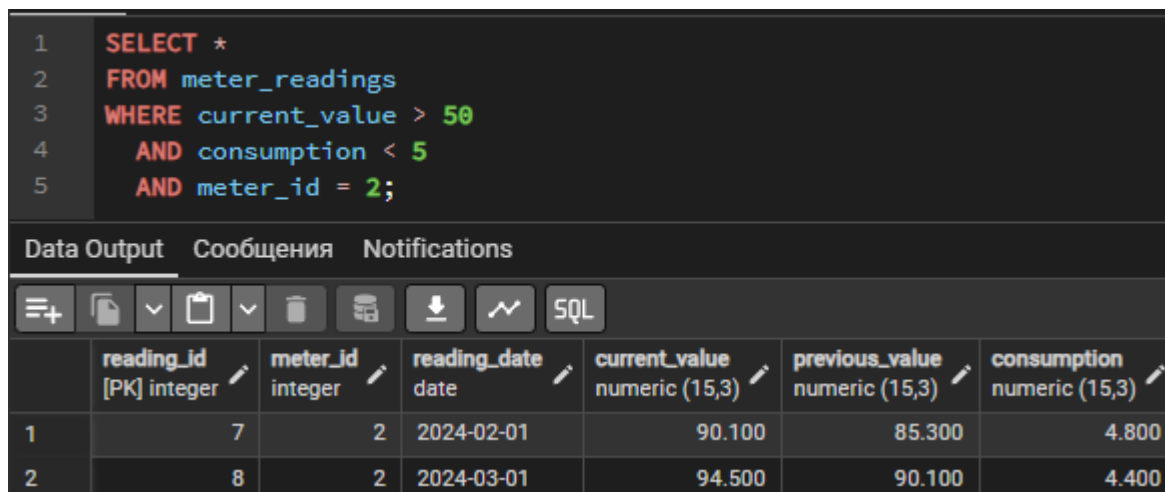
	reading_id [PK] integer	meter_id integer	reading_date date	current_value numeric (15,3)	previous_value numeric (15,3)	consumption numeric (15,3)
1	4	1	2024-03-01	137.800	131.200	6.600
2	12	3	2024-03-01	1450.000	1350.000	100.000
3	18	5	2024-03-01	5400.000	5200.000	200.000
4	21	6	2024-03-01	11000.000	10500.000	500.000
5	24	7	2024-03-01	171.200	160.500	10.700
6	30	9	2024-03-01	970.000	910.000	60.000
7	33	10	2024-03-01	108.200	101.500	6.700
8	39	13	2024-03-01	185.500	180.000	5.500
9	40	14	2024-03-01	650.000	620.000	30.000
10	41	15	2024-03-01	2200.000	2100.000	100.000

Рисунок 10 – Показания после 01.02.2024 с аномальными значениями

б3. Условия AND, AND:

Задача: Найти показания счётчиков, где текущее значение больше 50, расход очень мал (менее 5), и это показание относится к конкретному счётчику (ID: 2).

Результат выполнения запроса представлен на рисунке 11.



The screenshot shows a SQL query editor with the following query:

```
1 SELECT *
2 FROM meter_readings
3 WHERE current_value > 50
4 AND consumption < 5
5 AND meter_id = 2;
```

Below the query editor, there is a tab labeled "Data Output" which is active. It shows a table with 7 columns: reading_id, meter_id, reading_date, current_value, previous_value, and consumption. The table contains two rows of data.

	reading_id [PK] integer	meter_id integer	reading_date date	current_value numeric (15,3)	previous_value numeric (15,3)	consumption numeric (15,3)
1	7	2	2024-02-01	90.100	85.300	4.800
2	8	2	2024-03-01	94.500	90.100	4.400

Рисунок 11 – Показания счётчика 2: высокое значение, малый расход

6.1.3. Многотабличный запрос с вычисляемым полем и простым условием

Задача: Рассчитать общую сумму всех начислений для конкретного клиента объединив данные из трёх таблиц.

Результат выполнения запроса представлен на рисунке 12.

1	SELECT
2	c.full_name,
3	cntr.contract_number,
4	SUM(a.amount) as total_accrued
5	FROM clients c
6	JOIN contracts cntr ON c.client_id = cntr.client_id
7	JOIN accruals a ON cntr.contract_id = a.contract_id
8	WHERE c.client_id = 1
9	GROUP BY c.full_name, cntr.contract_number;

Data Output			
	full_name character varying (255)	contract_number character varying (50)	total_accrued numeric
1	Иванов Иван Иванович	ВОД-2023-001	1168.38
2	Иванов Иван Иванович	ЭЛ-2023-001	1845.00

Рисунок 12 – Сумма начислений по договорам клиента Иванова И.И.

6.1.4. Многотабличные запросы со сложными условиями

d1. Условия OR, AND, NOT:

Задача: Найти платежи, сделанные картой или наличными, исключая мелкие платежи.

Результат выполнения запроса представлен на рисунке 13.

1	SELECT p.*, c.full_name
2	FROM payments p
3	JOIN clients c ON p.client_id = c.client_id
4	WHERE (p.payment_method = 'Карта' OR p.payment_method = 'Наличные')
5	AND NOT p.amount < 500;

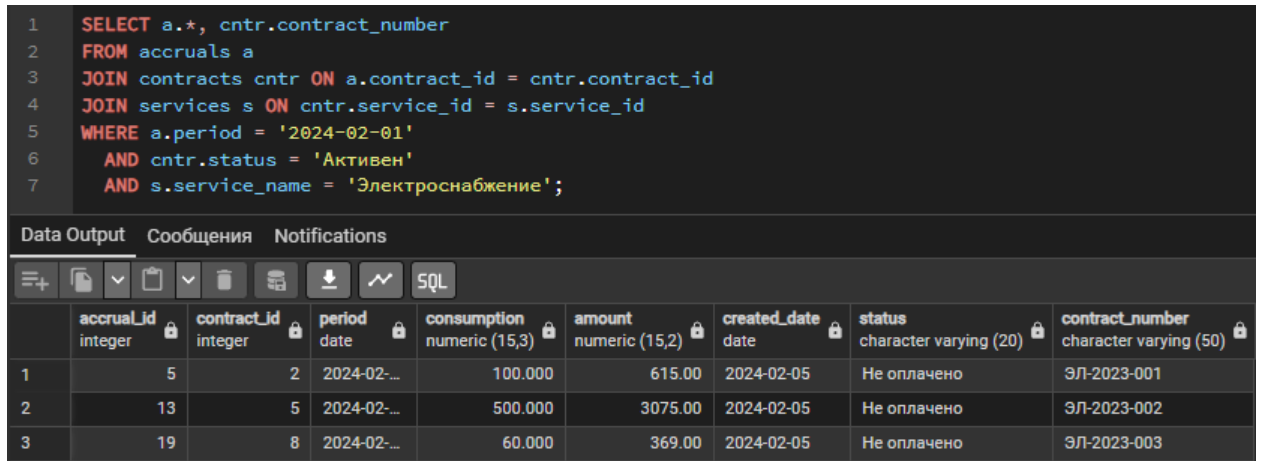
Data Output						
	payment_id integer	client_id integer	payment_date date	amount numeric (15,2)	payment_method character varying (50)	full_name character varying (255)
1	3	1	2024-03-20	500.00	Карта	Иванов Иван Иванович
2	8	3	2024-02-05	5000.00	Карта	ООО "Вектор"
3	10	4	2024-01-25	568.75	Карта	Сидоров Алексей Петров...
4	12	5	2024-01-15	1500.00	Карта	ИП Козлова М.В.

Рисунок 13 – Крупные платежи картой/наличными

d2. Условия AND, AND:

Задача: Сформировать реестр начислений за февраль 2024 года только по действующим договорам на электроснабжение.

Результат выполнения запроса представлен на рисунке 14.



```
1 SELECT a.*, cntr.contract_number
2 FROM accruals a
3 JOIN contracts cntr ON a.contract_id = cntr.contract_id
4 JOIN services s ON cntr.service_id = s.service_id
5 WHERE a.period = '2024-02-01'
6 AND cntr.status = 'Активен'
7 AND s.service_name = 'Электроснабжение';
```

Data Output Сообщения Notifications

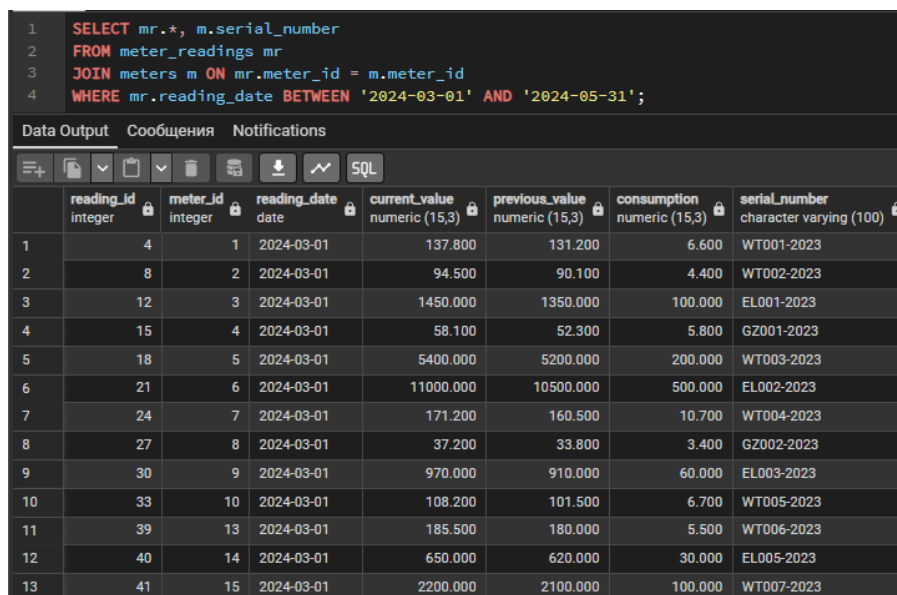
	accrual_id integer	contract_id integer	period date	consumption numeric (15,3)	amount numeric (15,2)	created_date date	status character varying (20)	contract_number character varying (50)
1	5	2	2024-02-...	100.000	615.00	2024-02-05	Не оплачено	ЭЛ-2023-001
2	13	5	2024-02-...	500.000	3075.00	2024-02-05	Не оплачено	ЭЛ-2023-002
3	19	8	2024-02-...	60.000	369.00	2024-02-05	Не оплачено	ЭЛ-2023-003

Рисунок 14 – Начисления за февраль 2024 по активным договорам на электричество

d3. Условие с оператором BETWEEN:

Задача: Получить все показания счётчиков, снятые во втором квартале 2024 года.

Результат выполнения запроса представлен на рисунке 15.



```
1 SELECT mr.*, m.serial_number
2 FROM meter_readings mr
3 JOIN meters m ON mr.meter_id = m.meter_id
4 WHERE mr.reading_date BETWEEN '2024-03-01' AND '2024-05-31';
```

Data Output Сообщения Notifications

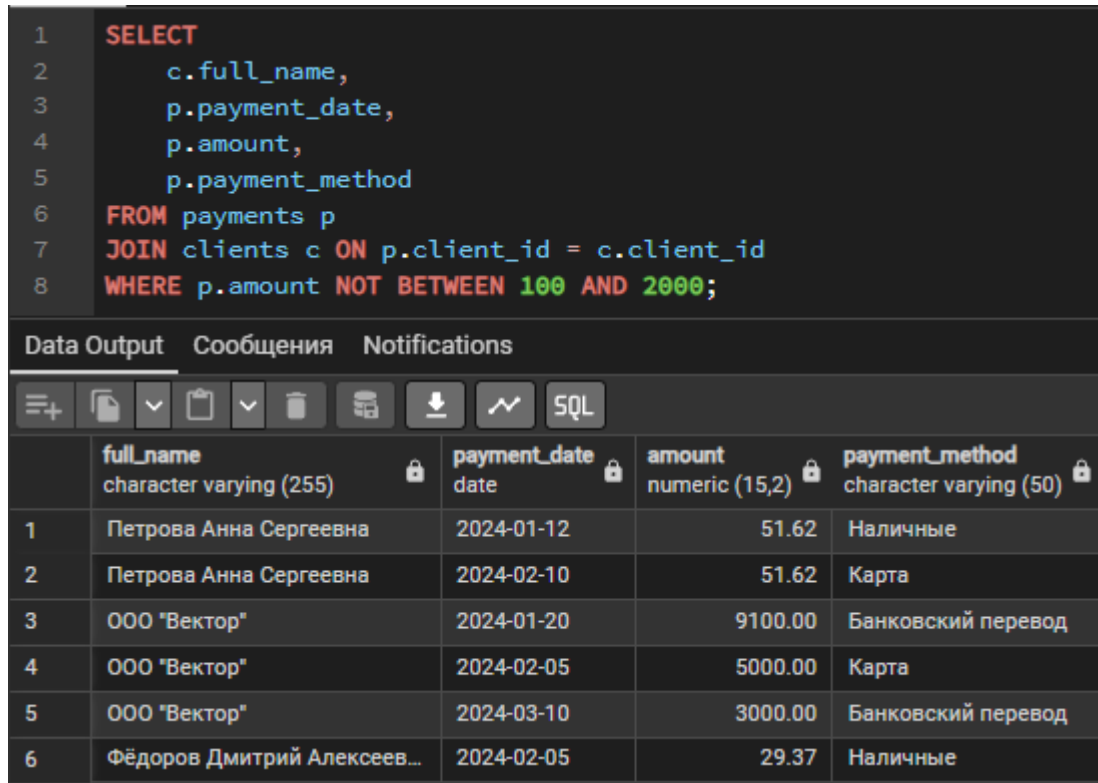
	reading_id integer	meter_id integer	reading_date date	current_value numeric (15,3)	previous_value numeric (15,3)	consumption numeric (15,3)	serial_number character varying (100)
1	4	1	2024-03-01	137.800	131.200	6.600	WT001-2023
2	8	2	2024-03-01	94.500	90.100	4.400	WT002-2023
3	12	3	2024-03-01	1450.000	1350.000	100.000	EL001-2023
4	15	4	2024-03-01	58.100	52.300	5.800	GZ001-2023
5	18	5	2024-03-01	5400.000	5200.000	200.000	WT003-2023
6	21	6	2024-03-01	11000.000	10500.000	500.000	EL002-2023
7	24	7	2024-03-01	171.200	160.500	10.700	WT004-2023
8	27	8	2024-03-01	37.200	33.800	3.400	GZ002-2023
9	30	9	2024-03-01	970.000	910.000	60.000	EL003-2023
10	33	10	2024-03-01	108.200	101.500	6.700	WT005-2023
11	39	13	2024-03-01	185.500	180.000	5.500	WT006-2023
12	40	14	2024-03-01	650.000	620.000	30.000	EL005-2023
13	41	15	2024-03-01	2200.000	2100.000	100.000	WT007-2023

Рисунок 15 – Показания счётчиков за II квартал 2024 года

d4. Условие с оператором NOT BETWEEN:

Задача: Найти клиентов и их платежи, где сумма платежа выходит за рамки диапазона (меньше 100 рублей или больше 2000 рублей).

Результат выполнения запроса представлен на рисунке 16.



The screenshot shows a SQL query in a dark-themed editor. The query selects client names, payment dates, amounts, and payment methods from a 'payments' table, joined with a 'clients' table. It filters for payments where the amount is not between 100 and 2000. Below the query, there's a toolbar with icons for file operations and a 'SQL' button. The results are shown in a table with 5 columns: an index, full_name, payment_date, amount, and payment_method.

```
1 SELECT
2     c.full_name,
3     p.payment_date,
4     p.amount,
5     p.payment_method
6 FROM payments p
7 JOIN clients c ON p.client_id = c.client_id
8 WHERE p.amount NOT BETWEEN 100 AND 2000;
```

	full_name character varying (255)	payment_date date	amount numeric (15,2)	payment_method character varying (50)
1	Петрова Анна Сергеевна	2024-01-12	51.62	Наличные
2	Петрова Анна Сергеевна	2024-02-10	51.62	Карта
3	ООО "Вектор"	2024-01-20	9100.00	Банковский перевод
4	ООО "Вектор"	2024-02-05	5000.00	Карта
5	ООО "Вектор"	2024-03-10	3000.00	Банковский перевод
6	Фёдоров Дмитрий Алексеев...	2024-02-05	29.37	Наличные

Рисунок 16 – Клиенты и их платежи с суммой вне диапазона 100–2000 руб.

d5. Условие с оператором IN:

Задача: Найти договоры для конкретных услуг (водоснабжение и электроснабжение) с информацией о клиентах для анализа портфеля договоров по ключевым услугам.

Результат выполнения запроса представлен на рисунке 17.

1

2

3

4

5

```
SELECT cntr.*, c.full_name, s.service_name
FROM contracts cntr
JOIN clients c ON cntr.client_id = c.client_id
JOIN services s ON cntr.service_id = s.service_id
WHERE s.service_name IN ('Водоснабжение', 'Электроснабжение');
```

Data Output

Сообщения

Notifications

SQL

	contract_id integer	contract_number character varying (50)	client_id integer	service_id integer	start_date date	end_date date	status character varying (20)	full_name character varying (255)	service_name character varying (100)
1	1	ВОД-2023-001	1	1	2023-01-15	[null]	Активен	Иванов Иван Иванович	Водоснабжение
2	2	ЭЛ-2023-001	1	2	2023-02-10	[null]	Активен	Иванов Иван Иванович	Электроснабжение
3	4	ВОД-2023-002	3	1	2023-01-20	2024-12-31	Активен	ООО "Вектор"	Водоснабжение
4	5	ЭЛ-2023-002	3	2	2023-02-15	[null]	Активен	ООО "Вектор"	Электроснабжение
5	6	ВОД-2023-003	4	1	2023-04-01	[null]	Активен	Сидоров Алексей Петрович	Водоснабжение
6	8	ЭЛ-2023-003	6	2	2023-06-01	[null]	Активен	Николаева Елена Викторовна	Электроснабжение
7	9	ВОД-2023-004	7	1	2023-07-05	[null]	Активен	Фёдоров Дмитрий Алексеевич	Водоснабжение
8	10	ЭЛ-2023-004	8	2	2023-08-12	[null]	Активен	Волкова Ольга Игоревна	Электроснабжение
9	12	ВОД-2023-005	10	1	2023-10-20	[null]	Активен	Киселёв Павел Викторович	Водоснабжение
10	13	ЭЛ-2023-005	11	2	2023-11-01	2024-10-31	Активен	Морозова Татьяна Сергеевна	Электроснабжение
11	14	ВОД-2023-006	12	1	2023-12-05	[null]	Расторгнут	ЗАО "ТехноПрофи"	Водоснабжение
12	15	ЭЛ-2023-006	13	2	2024-01-10	[null]	Активен	Белов Андрей Николаевич	Электроснабжение
13	17	ВОД-2023-007	15	1	2024-03-01	[null]	Активен	ИП Семёнов Р.К.	Водоснабжение
14	18	ЭЛ-2023-007	16	2	2024-03-10	[null]	Активен	Данилов Михаил Олегович	Электроснабжение
15	19	ВОД-2023-008	17	1	2024-04-05	[null]	Активен	Орлова Светлана Владимировна	Водоснабжение

Рисунок 17 – Договоры на водоснабжение и электроснабжение (с клиентами)

d6. Условие с оператором NOT IN:

Задача: Найти договоры, которые НЕ относятся к водоснабжению или газоснабжению (т.е. только электроснабжение), с информацией о клиентах для выделения отдельной категории договоров.

Результат выполнения запроса представлен на рисунке 18.

1

2

3

4

5

```
SELECT cntr.*, c.full_name, s.service_name
FROM contracts cntr
JOIN clients c ON cntr.client_id = c.client_id
JOIN services s ON cntr.service_id = s.service_id
WHERE s.service_name NOT IN ('Водоснабжение', 'Газоснабжение');
```

Data Output

Сообщения

Notifications

<

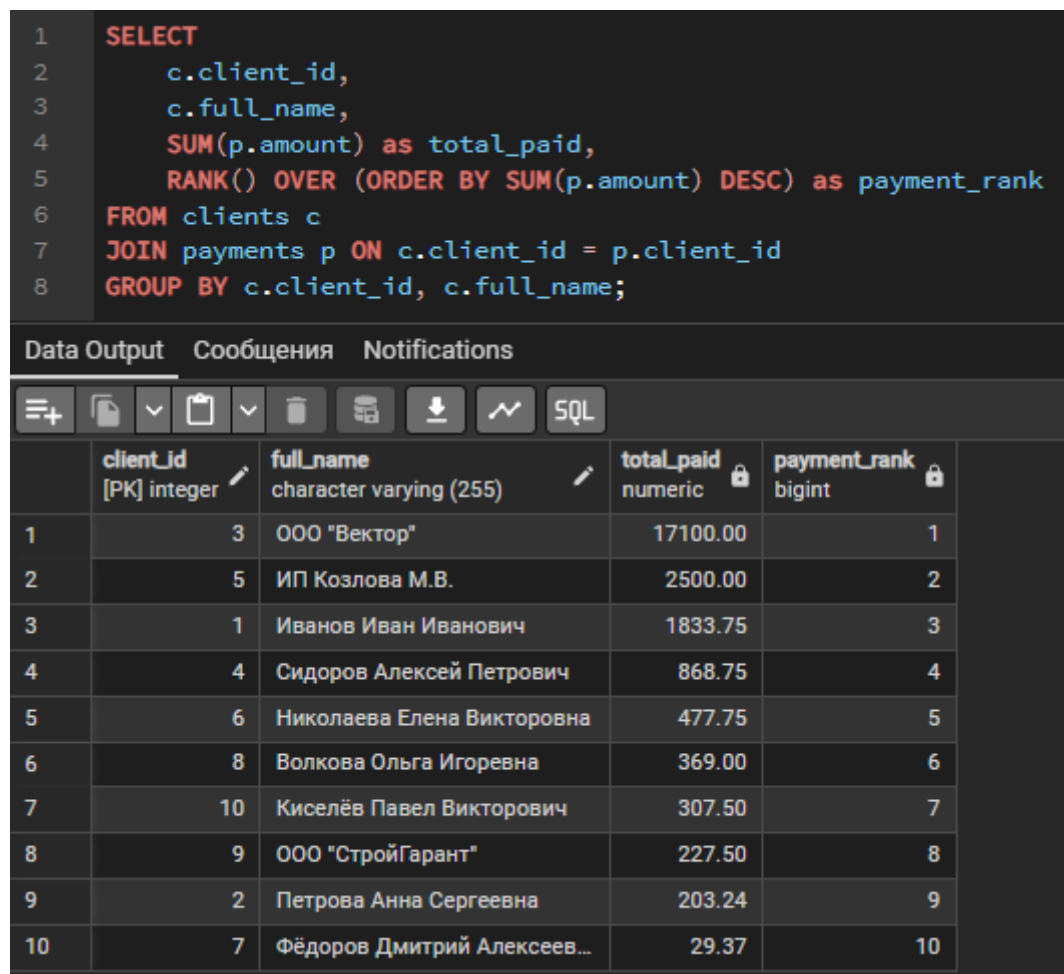
Рисунок 18 – Договоры на услуги, кроме водоснабжения и газоснабжения (с клиентами)

6.2. Запросы с использованием оконных функций

1. Ранжирование клиентов по платёжеспособности:

Задача: Проранжировать клиентов по общей сумме внесённых платежей для выделения наиболее платёжеспособных.

Результат выполнения запроса представлен на рисунке 19.



```
1 SELECT
2     c.client_id,
3     c.full_name,
4     SUM(p.amount) as total_paid,
5     RANK() OVER (ORDER BY SUM(p.amount) DESC) as payment_rank
6 FROM clients c
7 JOIN payments p ON c.client_id = p.client_id
8 GROUP BY c.client_id, c.full_name;
```

	client_id [PK] integer	full_name character varying (255)	total_paid numeric	payment_rank bigint
1	3	ООО "Вектор"	17100.00	1
2	5	ИП Козлова М.В.	2500.00	2
3	1	Иванов Иван Иванович	1833.75	3
4	4	Сидоров Алексей Петрович	868.75	4
5	6	Николаева Елена Викторовна	477.75	5
6	8	Волкова Ольга Игоревна	369.00	6
7	10	Киселёв Павел Викторович	307.50	7
8	9	ООО "СтройГарант"	227.50	8
9	2	Петрова Анна Сергеевна	203.24	9
10	7	Фёдоров Дмитрий Алексеев...	29.37	10

Рисунок 19 – Рейтинг клиентов по сумме оплат

2. Анализ динамики начислений по договорам:

Задача: Для каждого договора вывести начисления за 2024 год, добавив столбец с суммой предыдущего начисления.

Результат выполнения запроса представлен на рисунке 20.

1	SELECT			
2	contract_id,			
3	period,			
4	amount,			
5	LAG(amount) OVER (PARTITION BY contract_id ORDER BY period) as prev_amount			
6	FROM accruals			
7	WHERE EXTRACT(YEAR FROM period) = 2024;			

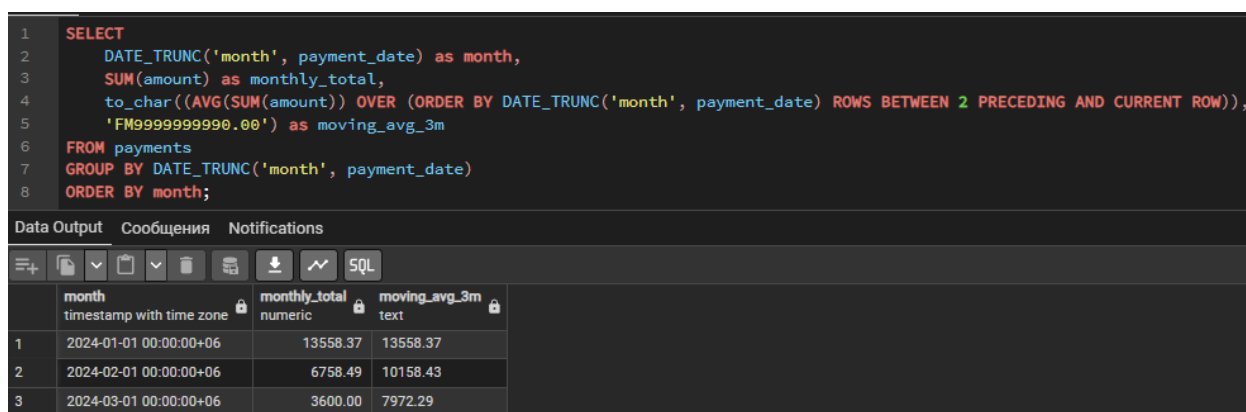
Data Output				
Сообщения				
Notifications				
+ 📄 ▼ 📋 ▼ 🗑️ 🔄 ⬇️ 📈 SQL				
	contract_id integer	period date	amount numeric (15,2)	prev_amount numeric
1	1	2024-01-01	568.75	[null]
2	1	2024-02-01	277.88	568.75
3	1	2024-03-01	321.75	277.88
4	1	2024-04-01	0.00	321.75
5	2	2024-01-01	615.00	[null]
6	2	2024-02-01	615.00	615.00
7	2	2024-03-01	615.00	615.00
8	2	2024-04-01	0.00	615.00
9	3	2024-01-01	51.62	[null]
10	3	2024-02-01	51.62	51.62
11	3	2024-03-01	51.62	51.62
12	3	2024-04-01	0.00	51.62
13	4	2024-01-01	9100.00	[null]
14	4	2024-02-01	9100.00	9100.00
15	4	2024-04-01	0.00	9100.00
16	5	2024-01-01	3075.00	[null]
17	5	2024-02-01	3075.00	3075.00
18	5	2024-04-01	0.00	3075.00
19	6	2024-01-01	477.75	[null]
20	6	2024-02-01	477.75	477.75
21	6	2024-04-01	0.00	477.75
22	7	2024-01-01	29.37	[null]
23	7	2024-02-01	29.37	29.37
24	7	2024-04-01	0.00	29.37
25	8	2024-01-01	369.00	[null]
26	8	2024-02-01	369.00	369.00
27	8	2024-04-01	0.00	369.00

Рисунок 20 – Динамика начислений по договорам

3. Расчёт скользящего среднего по платежам:

Задача: Рассчитать скользящее среднее суммы поступивших платежей за 3 месяца.

Результат выполнения запроса представлен на рисунке 21.



The screenshot shows a SQL query editor with the following query:

```
1 SELECT
2   DATE_TRUNC('month', payment_date) as month,
3   SUM(amount) as monthly_total,
4   to_char((AVG(SUM(amount)) OVER (ORDER BY DATE_TRUNC('month', payment_date) ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)),
5   'FM9999999990.00') as moving_avg_3m
6 FROM payments
7 GROUP BY DATE_TRUNC('month', payment_date)
8 ORDER BY month;
```

Below the query, there is a table with the following data:

	month timestamp with time zone	monthly_total numeric	moving_avg_3m text
1	2024-01-01 00:00:00+06	13558.37	13558.37
2	2024-02-01 00:00:00+06	6758.49	10158.43
3	2024-03-01 00:00:00+06	3600.00	7972.29

Рисунок 21 – Скользящее среднее месячных платежей

6.3. Запросы с вычисляемыми полями

1. Расчёт пени по просроченным начислениям:

Задача: Для каждого начисления рассчитать размер пени (0.5% от суммы за каждый день просрочки, начиная с 25-го числа месяца начисления).

Результат выполнения запроса представлен на рисунке 22.

1	SELECT
2	accrual_id,
3	contract_id,
4	period,
5	amount,
6	status,
7	CASE
8	WHEN status != 'Оплачено' AND CURRENT_DATE > (period + INTERVAL '25 days')
9	THEN ROUND((amount * 0.005 * EXTRACT(DAY FROM CURRENT_DATE - (period + INTERVAL '25 days')))::numeric, 2)
10	ELSE 0
11	END as penalty
12	FROM accruals;

	accrual_id [PK] integer	contract_id integer	period date	amount numeric (15,2)	status character varying (20)	penalty numeric
1	1	1	2024-01-...	568.75	Оплачено	0
2	2	1	2024-02-...	277.88	Частично оплачено	919.78
3	3	1	2024-03-...	321.75	Не оплачено	1018.34
4	4	2	2024-01-...	615.00	Оплачено	0
5	5	2	2024-02-...	615.00	Не оплачено	2035.65
6	6	2	2024-03-...	615.00	Не оплачено	1946.48
7	7	3	2024-01-...	51.62	Оплачено	0
8	8	3	2024-02-...	51.62	Оплачено	0
9	9	3	2024-03-...	51.62	Не оплачено	163.38
10	10	4	2024-01-...	9100.00	Оплачено	0
11	11	4	2024-02-...	9100.00	Не оплачено	30121.00
12	12	5	2024-01-...	3075.00	Частично оплачено	10654.88
13	13	5	2024-02-...	3075.00	Не оплачено	10178.25
14	14	6	2024-01-...	477.75	Оплачено	0
15	15	6	2024-02-...	477.75	Не оплачено	1581.35
16	16	7	2024-01-...	29.37	Оплачено	0
17	17	7	2024-02-...	29.37	Не оплачено	97.21
18	18	8	2024-01-...	369.00	Оплачено	0
19	19	8	2024-02-...	369.00	Не оплачено	1221.39
20	20	9	2024-01-...	227.50	Оплачено	0
21	21	10	2024-01-...	307.50	Оплачено	0
22	22	11	2024-01-...	24.92	Оплачено	0
23	23	12	2024-02-...	295.75	Не оплачено	978.93

Рисунок 22 – Расчёт пени по просроченным начислениям

2. Категоризация клиентов по объёму платежей:

Задача: Классифицировать клиентов на категории "Малый", "Средний" и "Крупный" в зависимости от общей суммы их платежей.

Результат выполнения запроса представлен на рисунке 23.

1	SELECT
2	c.client_id,
3	c.full_name,
4	c.client_type,
5	COALESCE(SUM(p.amount), 0) as total_paid,
6	CASE
7	WHEN COALESCE(SUM(p.amount), 0) < 1000 THEN 'Малый'
8	WHEN COALESCE(SUM(p.amount), 0) BETWEEN 1000 AND 5000 THEN 'Средний'
9	ELSE 'Крупный'
10	END as payment_category
11	FROM clients c
12	LEFT JOIN payments p ON c.client_id = p.client_id
13	GROUP BY c.client_id, c.full_name, c.client_type;

Data Output	Сообщения	Notifications
-------------	-----------	---------------

≡+	📄	▼	🗑️	🗑️	📥	📶	SQL
----	---	---	----	----	---	---	-----

	client_id [PK] integer	full_name character varying (255)	client_type character varying (20)	total_paid numeric	payment_category text
1	4	Сидоров Алексей Петрович	Физическое лицо	868.75	Малый
2	10	Киселёв Павел Викторович	Физическое лицо	307.50	Малый
3	6	Николаева Елена Викторовна	Физическое лицо	477.75	Малый
4	14	Григорьева Ирина Петровна	Физическое лицо	0	Малый
5	13	Белов Андрей Николаевич	Физическое лицо	0	Малый
6	2	Петрова Анна Сергеевна	Физическое лицо	203.24	Малый
7	16	Данилов Михаил Олегович	Физическое лицо	0	Малый
8	11	Морозова Татьяна Сергеевна	Физическое лицо	0	Малый
9	9	ООО "СтройГарант"	Юридическое лицо	227.50	Малый
10	7	Фёдоров Дмитрий Алексеевич	Физическое лицо	29.37	Малый
11	15	ИП Семёнов Р.К.	Юридическое лицо	0	Малый
12	12	ЗАО "ТехноПрофи"	Юридическое лицо	0	Малый
13	3	ООО "Вектор"	Юридическое лицо	17100.00	Крупный
14	17	Орлова Светлана Владимиров...	Физическое лицо	0	Малый
15	1	Иванов Иван Иванович	Физическое лицо	1833.75	Средний
16	5	ИП Козлова М.В.	Юридическое лицо	2500.00	Средний
17	18	Титов Виктор Александрович	Физическое лицо	0	Малый
18	8	Волкова Ольга Игоревна	Физическое лицо	369.00	Малый

Рисунок 23 – Категоризация клиентов по объёму платежей

6.4. Запросы с агрегатными функциями

1. Анализ популярности способов оплаты:

Задача: Подсчитать, каким способом оплаты клиенты вносят средства чаще всего, и какую общую сумму приносит каждый способ.

Результат выполнения запроса представлен на рисунке 24.

1	SELECT
2	payment_method,
3	COUNT(*) as payment_count,
4	SUM(amount) as total_amount
5	FROM payments
6	GROUP BY payment_method
7	ORDER BY total_amount DESC;

Data Output		Сообщения	Notifications
	payment_method character varying (50)	payment_count bigint	total_amount numeric
1	Банковский перевод	6	14691.25
2	Карта	9	8844.62
3	Наличные	3	380.99

Рисунок 24 – Статистика по способам оплаты

2. Выявление клиентов с задолженностью за 2024 год:

Задача: Определить список клиентов, у которых сумма начислений за 2024 год превысила сумму оплат.

Результат выполнения запроса представлен на рисунке 25.

1	SELECT
2	c.client_type,
3	c.full_name,
4	COALESCE(SUM(a.amount), 0) as total_accrued_2024,
5	COALESCE(SUM(p.amount), 0) as total_paid_2024,
6	(COALESCE(SUM(a.amount), 0) - COALESCE(SUM(p.amount), 0)) as debt_2024
7	FROM clients c
8	LEFT JOIN contracts cntr ON c.client_id = cntr.client_id
9	LEFT JOIN accruals a ON cntr.contract_id = a.contract_id AND EXTRACT(YEAR FROM a.period) = 2024
10	LEFT JOIN payments p ON c.client_id = p.client_id AND EXTRACT(YEAR FROM p.payment_date) = 2024
11	GROUP BY c.client_id, c.client_type, c.full_name
12	HAVING (COALESCE(SUM(a.amount), 0) - COALESCE(SUM(p.amount), 0)) > 0
13	ORDER BY c.client_type, debt_2024 DESC, c.full_name;

Data Output		Сообщения	Notifications
	client_type character varying (20)	full_name character varying (255)	total_accrued_2024 numeric
			total_paid_2024 numeric
			debt_2024 numeric
1	Физическое лицо	Морозова Татьяна Сергеевна	184.50
2	Физическое лицо	Фёдоров Дмитрий Алексеев...	227.50
			58.74
			168.76

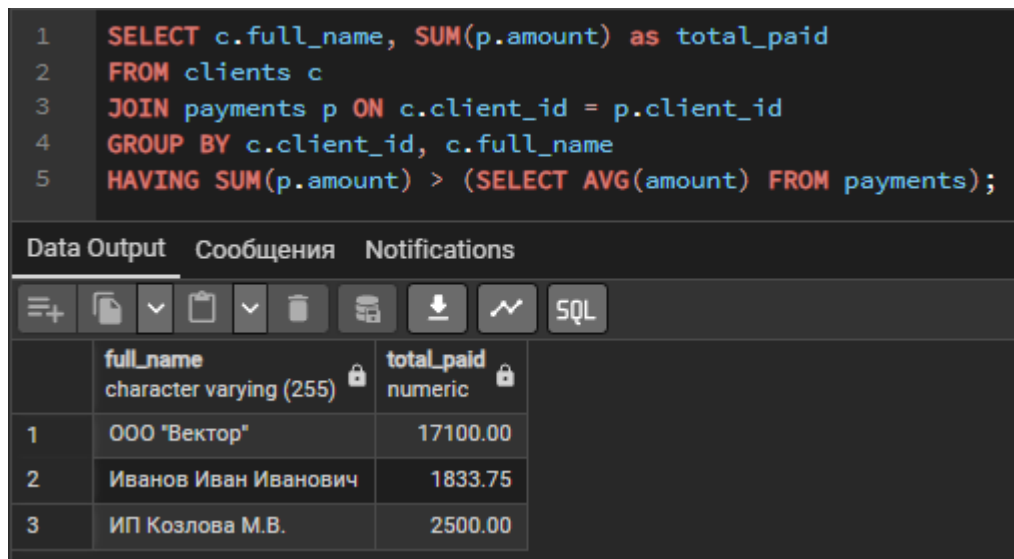
Рисунок 25 – Список клиентов с задолженностью за 2024 год

6.5. Запросы с использованием подзапросов

1. Запрос с использованием подзапроса, возвращающего единичное значение

Задача: Найти клиентов, чей общий размер платежей больше среднего платежа по всем клиентам.

Результат выполнения запроса представлен на рисунке 26.



```
1 SELECT c.full_name, SUM(p.amount) as total_paid
2 FROM clients c
3 JOIN payments p ON c.client_id = p.client_id
4 GROUP BY c.client_id, c.full_name
5 HAVING SUM(p.amount) > (SELECT AVG(amount) FROM payments);
```

	full_name character varying (255)	total_paid numeric
1	ООО "Вектор"	17100.00
2	Иванов Иван Иванович	1833.75
3	ИП Козлова М.В.	2500.00

Рисунок 26 – Запрос с использованием подзапроса, возвращающего единичное значение

2. Запрос с использованием операций IN и NOT IN

Задача: Найти клиентов, у которых есть договоры на водоснабжение (IN), но нет договоров на газоснабжение (NOT IN).

Результат выполнения запроса представлен на рисунке 27.

```

1  SELECT c.client_id, c.full_name
2  FROM clients c
3  WHERE c.client_id IN (
4      SELECT cntr.client_id
5      FROM contracts cntr
6      JOIN services s ON cntr.service_id = s.service_id
7      WHERE s.service_name = 'Водоснабжение'
8  )
9  AND c.client_id NOT IN (
10     SELECT cntr.client_id
11     FROM contracts cntr
12     JOIN services s ON cntr.service_id = s.service_id
13     WHERE s.service_name = 'Газоснабжение'
14 );

```

Data Output Сообщения Notifications

	client_id [PK] integer	full_name character varying (255)
1	10	Киселёв Павел Викторович
2	7	Фёдоров Дмитрий Алексеевич
3	1	Иванов Иван Иванович
4	4	Сидоров Алексей Петрович
5	15	ИП Семёнов Р.К.
6	12	ЗАО "ТехноПрофи"
7	3	ООО "Вектор"
8	17	Орлова Светлана Владимиров...

Рисунок 27 – Запрос с использованием IN и NOT IN

3. Запрос с использованием ключевых слов ANY и ALL

Задача: Найти договоры, у которых сумма начислений больше ЛЮБОГО начисления по договору 1 (ANY) и при этом больше ВСЕХ начислений по договору 2 (ALL).

Результат выполнения запроса представлен на рисунке 28.

```

1 SELECT cntr.contract_id, cntr.contract_number, SUM(a.amount) as total_accrued
2 FROM contracts cntr
3 JOIN accruals a ON cntr.contract_id = a.contract_id
4 GROUP BY cntr.contract_id, cntr.contract_number
5 HAVING SUM(a.amount) > ANY (
6     SELECT amount FROM accruals WHERE contract_id = 1
7 )
8 AND SUM(a.amount) > ALL (
9     SELECT amount FROM accruals WHERE contract_id = 2
10 );

```

Data Output Сообщения Notifications

	contractId [PK] integer	contract_number character varying (50)	total_accrued numeric
1	2	ЭЛ-2023-001	1845.00
2	1	ВОД-2023-001	1168.38
3	4	ВОД-2023-002	18200.00
4	6	ВОД-2023-003	955.50
5	8	ЭЛ-2023-003	738.00
6	5	ЭЛ-2023-002	6150.00

Рисунок 28 – Запрос с использованием ключевых слов ANY и ALL

4. Запрос с использованием операций EXISTS и NOT EXISTS

Задача: Найти услуги, для которых существуют действующие тарифы (EXISTS) но нет просроченных начислений (NOT EXISTS).

Результат выполнения запроса представлен на рисунке 29.

```

1  SELECT s.service_id, s.service_name
2  FROM services s
3  WHERE EXISTS (
4      SELECT 1
5      FROM tariffs t
6      WHERE t.service_id = s.service_id
7      AND t.start_date <= CURRENT_DATE
8      AND t.end_date >= CURRENT_DATE
9  )
10 AND NOT EXISTS (
11     SELECT 1
12     FROM contracts cntr
13     JOIN accruals a ON cntr.contract_id = a.contract_id
14     WHERE cntr.service_id = s.service_id
15     AND a.status = 'Просрочено'
16 );

```

Data Output Сообщения Notifications

	service_id [PK] integer	service_name character varying (100)
1	2	Электроснабжение
2	3	Газоснабжение

Рисунок 29 – Запрос с использованием операций EXISTS и NOT EXISTS

6.6. Запрос на основе материализованного представления

Задача: Создать материализованное представление (рис. 30) для быстрого доступа к балансу клиентов, а затем выполнить к нему запрос для получения списка клиентов с задолженностью.

Результат выполнения запроса представлен на рисунке 31.

```

1 DROP MATERIALIZED VIEW IF EXISTS mv_client_balance;
2
3 CREATE MATERIALIZED VIEW mv_client_balance AS
4 SELECT
5     c.client_id,
6     c.full_name,
7     c.client_type,
8     COALESCE(SUM(a.amount), 0) as total_accrued,
9     COALESCE(SUM(p.amount), 0) as total_paid,
10    (COALESCE(SUM(a.amount), 0) - COALESCE(SUM(p.amount), 0)) as current_balance
11 FROM clients c
12 LEFT JOIN contracts cntr ON c.client_id = cntr.client_id
13 LEFT JOIN accruals a ON cntr.contract_id = a.contract_id
14 LEFT JOIN payments p ON c.client_id = p.client_id
15 GROUP BY c.client_id, c.full_name, c.client_type;

```

Рисунок 30 – Создание материализованного представления mv_client_balance

```

1 SELECT *
2 FROM mv_client_balance
3 WHERE current_balance > 0
4 ORDER BY current_balance DESC;

```

	client_id integer	full_name character varying (255)	client_type character varying (20)	total_accrued numeric	total_paid numeric	current_balance numeric
1	11	Морозова Татьяна Сергеевна	Физическое лицо	184.50	0	184.50
2	7	Фёдоров Дмитрий Алексеев...	Физическое лицо	227.50	58.74	168.76

Рисунок 31 – Клиенты с задолженностью (запрос к материализованному представлению)

7. Разработка объектов БД

7.1. Разработка материализованных представлений

1. В пункте 6.6. уже было создано одно материализованное представление mv_client_balance (Рисунок 30).
2. материализованное представление: сводка по договорам

Задача: Создать материализованное представление mv_contract_summary, которое агрегирует ключевую информацию по договорам: данные клиента, услугу, общую сумму начислений и оплат по договору.

Скрипт для создания материализованного представления представлен на рисунке 32.

```

1 DROP MATERIALIZED VIEW IF EXISTS mv_contract_summary;
2
3 CREATE MATERIALIZED VIEW mv_contract_summary AS
4 SELECT
5     cntr.contract_id,
6     cntr.contract_number,
7     c.full_name AS client_name,
8     c.client_type,
9     s.service_name,
10    cntr.start_date,
11    cntr.end_date,
12    cntr.status AS contract_status,
13    COALESCE(SUM(a.amount), 0) AS total_accrued,
14    COALESCE(SUM(pa.allocated_amount), 0) AS total_paid,
15    (COALESCE(SUM(a.amount), 0) - COALESCE(SUM(pa.allocated_amount), 0)) AS contract_balance
16 FROM contracts cntr
17 JOIN clients c ON cntr.client_id = c.client_id
18 JOIN services s ON cntr.service_id = s.service_id
19 LEFT JOIN accruals a ON cntr.contract_id = a.contract_id
20 LEFT JOIN payment_allocations pa ON a.accrual_id = pa.accrual_id
21 GROUP BY cntr.contract_id, cntr.contract_number, c.full_name, c.client_type,
22          s.service_name, cntr.start_date, cntr.end_date, cntr.status;

```

Рисунок 32. – Создание материализованного представления
mv_contract_summary

Результат выполнения запроса представлен на рисунке 33.

```

1 SELECT *
2 FROM mv_contract_summary
3 WHERE contract_balance > 0
4 ORDER BY contract_balance DESC;

```

contract_id	contract_number	client_name	client_type	service_name	start_date	end_date	contract_status	total_accrued	total_paid	contract_balance
4	ВОД-2023-002	ООО "Вектор"	Юридическое лицо	Водоснабжение	2023-01-20	2024-12-31	Активен	27300.00	17100.00	10200.00
5	ЭЛ-2023-002	ООО "Вектор"	Юридическое лицо	Электроснабжение	2023-02-15	[null]	Активен	6150.00	1052.75	5097.25
2	ЭЛ-2023-001	Иванов Иван Иванович	Физическое лицо	Электроснабжение	2023-02-10	[null]	Активен	2460.00	1035.50	1424.50
1	ВОД-2023-001	Иванов Иван Иванович	Физическое лицо	Водоснабжение	2023-01-15	[null]	Активен	1446.26	846.63	599.63
8	ЭЛ-2023-003	Николаева Елена Викторов...	Физическое лицо	Электроснабжение	2023-06-01	[null]	Активен	738.00	398.37	339.63
12	ВОД-2023-005	Киселёв Павел Викторович	Физическое лицо	Водоснабжение	2023-10-20	[null]	Активен	295.75	0	295.75
13	ЭЛ-2023-005	Морозова Татьяна Сергеев...	Физическое лицо	Электроснабжение	2023-11-01	2024-10-31	Активен	184.50	0	184.50
7	ГАЗ-2023-002	ИП Козлова М.В.	Юридическое лицо	Газоснабжение	2023-05-10	[null]	Активен	58.74	0	58.74

Рисунок 33 – Использование материализованного представления
mv_contract_summary

7.2. Разработка курсоров

1. Курсор для получения списка просроченных начислений

Задача: Создать курсор для последовательной обработки просроченных начислений и формирования списка клиентов с задолженностью.

Результат выполнения запроса представлен на рисунке 34.

```
1 DO $$
2 DECLARE
3   cur_rec RECORD;
4   cur_cursor CURSOR FOR
5     SELECT
6       c.full_name,
7       c.phone,
8       c.email,
9       c.address,
10      a.accrual_id,
11      a.period,
12      a.amount
13   FROM accruals a
14   JOIN contracts cntr ON a.contract_id = cntr.contract_id
15   JOIN clients c ON cntr.client_id = c.client_id
16   WHERE a.status != 'Оплачено'
17   AND a.period < CURRENT_DATE - INTERVAL '30 days'
18   ORDER BY a.period;
19 BEGIN
20   OPEN cur_cursor;
21   LOOP
22     FETCH cur_cursor INTO cur_rec;
23     EXIT WHEN NOT FOUND;
24
25     RAISE NOTICE 'Клиент: %, Телефон: %, Email: %, Адрес: %, Начисление ID: %, Период: %, Сумма: % руб.',
26       cur_rec.full_name,
27       cur_rec.phone,
28       COALESCE(cur_rec.email, 'не указан'),
29       COALESCE(cur_rec.address, 'не указан'),
30       cur_rec.accrual_id,
31       cur_rec.period,
32       cur_rec.amount;
33
34   END LOOP;
35   CLOSE cur_cursor;
36 END $$;
```

Data Output Messages Notifications

NOTICE: Клиент: ООО "Вектор", Телефон: +7-495-123-45-67, Email: director@vector.ru, Адрес: г. Москва, ул. Тверская, д. 10, офис 305, Начисление ID: 12, Период: 2024-01-01, Сумма: 3075.00 руб.

NOTICE: Клиент: Иванов Иван Иванович, Телефон: +7-900-123-45-67, Email: ivanov@mail.ru, Адрес: г. Омск, ул. Ленина, д. 1, кв. 5, Начисление ID: 2, Период: 2024-02-01, Сумма: 277.88 руб.

NOTICE: Клиент: Иванов Иван Иванович, Телефон: +7-900-123-45-67, Email: ivanov@mail.ru, Адрес: г. Омск, ул. Ленина, д. 1, кв. 5, Начисление ID: 5, Период: 2024-02-01, Сумма: 615.00 руб.

NOTICE: Клиент: ООО "Вектор", Телефон: +7-495-123-45-67, Email: director@vector.ru, Адрес: г. Москва, ул. Тверская, д. 10, офис 305, Начисление ID: 11, Период: 2024-02-01, Сумма: 9100.00 руб.

NOTICE: Клиент: ООО "Вектор", Телефон: +7-495-123-45-67, Email: director@vector.ru, Адрес: г. Москва, ул. Тверская, д. 10, офис 305, Начисление ID: 13, Период: 2024-02-01, Сумма: 3075.00 руб.

NOTICE: Клиент: Сидоров Алексей Петрович, Телефон: +7-902-345-67-89, Email: не указан, Адрес: г. Омск, ул. Кирова, д. 15, кв. 8, Начисление ID: 15, Период: 2024-02-01, Сумма: 477.75 руб.

NOTICE: Клиент: ИП Козлова М.В., Телефон: +7-916-456-78-90, Email: kozlova@mail.ru, Адрес: г. Омск, ул. 10 лет Октября, д. 5, Начисление ID: 17, Период: 2024-02-01, Сумма: 29.37 руб.

NOTICE: Клиент: Николаева Елена Викторовна, Телефон: +7-903-456-78-91, Email: не указан, Адрес: г. Омск, ул. Маршала Жукова, д. 30, кв. 45, Начисление ID: 19, Период: 2024-02-01, Сумма: 369.00 руб.

NOTICE: Клиент: Киселев Павел Викторович, Телефон: +7-906-789-01-24, Email: не указан, Адрес: г. Омск, ул. Декабристов, д. 12, кв. 9, Начисление ID: 23, Период: 2024-02-01, Сумма: 285.75 руб.

NOTICE: Клиент: Иванов Иван Иванович, Телефон: +7-900-123-45-67, Email: ivanov@mail.ru, Адрес: г. Омск, ул. Ленина, д. 1, кв. 5, Начисление ID: 3, Период: 2024-03-01, Сумма: 321.75 руб.

NOTICE: Клиент: Иванов Иван Иванович, Телефон: +7-900-123-45-67, Email: ivanov@mail.ru, Адрес: г. Омск, ул. Ленина, д. 1, кв. 5, Начисление ID: 6, Период: 2024-03-01, Сумма: 615.00 руб.

NOTICE: Клиент: Петрова Анна Сергеевна, Телефон: +7-901-234-56-78, Email: petrova@yandex.ru, Адрес: г. Омск, пр. Мира, д. 25, кв. 12, Начисление ID: 9, Период: 2024-03-01, Сумма: 51.82 руб.

NOTICE: Клиент: Морозова Татьяна Сергеевна, Телефон: +7-907-890-12-35, Email: не указан, Адрес: г. Омск, ул. Герцена, д. 5, кв. 33, Начисление ID: 24, Период: 2024-03-01, Сумма: 184.50 руб.

Рисунок 34 – Выполнение курсора для выборки просроченных начислений

2. Курсор для формирования ежемесячного отчета по потреблению услуг по клиентам

Задача: Создать курсор для формирования детального отчета о потреблении услуг за указанный месяц для всех активных договоров.

Результат выполнения запроса представлен на рисунке 35.

```

1 DO $$
2 DECLARE
3     report_rec RECORD;
4     report_cursor CURSOR(year_month DATE) FOR
5         SELECT
6             c.full_name AS client_name,
7             s.service_name,
8             cntr.contract_number,
9             m.serial_number AS meter_serial,
10            mr.reading_date,
11            mr.consumption,
12            t.rate,
13            (mr.consumption * t.rate) AS calculated_cost
14        FROM meter_readings mr
15        JOIN meters m ON mr.meter_id = m.meter_id
16        JOIN contracts cntr ON m.contract_id = cntr.contract_id
17        JOIN clients c ON cntr.client_id = c.client_id
18        JOIN services s ON cntr.service_id = s.service_id
19        JOIN tariffs t ON s.service_id = t.service_id
20        WHERE DATE_TRUNC('month', mr.reading_date) = DATE_TRUNC('month', year_month)
21              AND mr.reading_date BETWEEN t.start_date AND t.end_date
22              AND cntr.status = 'Активен'
23        ORDER BY c.full_name, s.service_name, mr.reading_date;
24 BEGIN
25     OPEN report_cursor('2024-03-01');
26     LOOP
27         FETCH report_cursor INTO report_rec;
28         EXIT WHEN NOT FOUND;
29
30         RAISE NOTICE 'Клиент: %, Услуга: %, Договор: %, Счетчик: %, Дата: %, Расход: %, Тариф: %, Сумма: % руб.',
31             report_rec.client_name,
32             report_rec.service_name,
33             report_rec.contract_number,
34             report_rec.meter_serial,
35             report_rec.reading_date,
36             to_char(report_rec.consumption, 'FM9999999990.00'),
37             report_rec.rate,
38             to_char(report_rec.calculated_cost, 'FM9999999990.00');
39     END LOOP;
40     CLOSE report_cursor;
41 END $$;

```

Data Output
Сообщения
Notifications

```

NOTICE: Клиент: Иванов Иван Иванович, Услуга: Водоснабжение, Договор: Б0Д-2023-001, Счетчик: WT002-2023, Дата: 2024-03-01, Расход: 4.40, Тариф: 45.50, Сумма: 200.20 руб.
NOTICE: Клиент: Иванов Иван Иванович, Услуга: Водоснабжение, Договор: Б0Д-2023-001, Счетчик: WT001-2023, Дата: 2024-03-01, Расход: 6.60, Тариф: 45.50, Сумма: 300.30 руб.
NOTICE: Клиент: Иванов Иван Иванович, Услуга: Электроснабжение, Договор: ЭЛ-2023-001, Счетчик: EL001-2023, Дата: 2024-03-01, Расход: 100.00, Тариф: 6.15, Сумма: 615.00 руб.
NOTICE: Клиент: ИП Козлова М.В., Услуга: Газоснабжение, Договор: ГАЗ-2023-002, Счетчик: GZ002-2023, Дата: 2024-03-01, Расход: 3.40, Тариф: 8.90, Сумма: 30.26 руб.
NOTICE: Клиент: Киселев Павел Викторович, Услуга: Водоснабжение, Договор: Б0Д-2023-005, Счетчик: WT006-2023, Дата: 2024-03-01, Расход: 5.50, Тариф: 45.50, Сумма: 250.25 руб.
NOTICE: Клиент: Морозова Татьяна Сергеевна, Услуга: Электроснабжение, Договор: ЭЛ-2023-005, Счетчик: EL005-2023, Дата: 2024-03-01, Расход: 30.00, Тариф: 6.15, Сумма: 184.50 руб.
NOTICE: Клиент: Николаева Елена Викторовна, Услуга: Электроснабжение, Договор: ЭЛ-2023-003, Счетчик: EL003-2023, Дата: 2024-03-01, Расход: 60.00, Тариф: 6.15, Сумма: 369.00 руб.
NOTICE: Клиент: ООО "Вектор", Услуга: Водоснабжение, Договор: Б0Д-2023-002, Счетчик: WT003-2023, Дата: 2024-03-01, Расход: 200.00, Тариф: 45.50, Сумма: 9100.00 руб.
NOTICE: Клиент: ООО "Вектор", Услуга: Электроснабжение, Договор: ЭЛ-2023-002, Счетчик: EL002-2023, Дата: 2024-03-01, Расход: 500.00, Тариф: 6.15, Сумма: 3075.00 руб.
NOTICE: Клиент: Петрова Анна Сергеевна, Услуга: Газоснабжение, Договор: ГАЗ-2023-001, Счетчик: GZ001-2023, Дата: 2024-03-01, Расход: 5.80, Тариф: 8.90, Сумма: 51.62 руб.
NOTICE: Клиент: Сидоров Алексей Петрович, Услуга: Водоснабжение, Договор: Б0Д-2023-003, Счетчик: WT004-2023, Дата: 2024-03-01, Расход: 10.70, Тариф: 45.50, Сумма: 486.65 руб.
NOTICE: Клиент: Фёдоров Дмитрий Алексеевич, Услуга: Водоснабжение, Договор: Б0Д-2023-004, Счетчик: WT005-2023, Дата: 2024-03-01, Расход: 6.70, Тариф: 45.50, Сумма: 304.65 руб.

```

Рисунок 35 – Выполнение курсора для формирования отчета по потреблению за март 2024

7.3. Разработка функции

1. Функция для расчёта задолженности клиента

Задача: Создать функцию для расчёта полной суммы задолженности клиента на заданную дату.

Результат выполнения запроса представлен на рисунке 36.

```

1 CREATE OR REPLACE FUNCTION get_client_debt(client_id INTEGER, target_date DATE)
2 RETURNS DECIMAL(15,2) AS $$
3 DECLARE
4     total_debt DECIMAL(15,2);
5 BEGIN
6     SELECT COALESCE(SUM(a.amount), 0) - COALESCE(SUM(pa.allocated_amount), 0)
7     INTO total_debt
8     FROM contracts cntr
9     LEFT JOIN accruals a ON cntr.contract_id = a.contract_id
10    LEFT JOIN payment_allocations pa ON a.accrual_id = pa.accrual_id
11    WHERE cntr.client_id = $1
12    AND a.period <= $2
13    AND a.status != 'Оплачено';
14
15    RETURN total_debt;
16 END;
17 $$ LANGUAGE plpgsql;
18
19 SELECT full_name, get_client_debt(1, '2024-03-31') AS debt_on_2024_03_31
20 FROM clients WHERE client_id = 1;

```

Data Output Сообщения Notifications

	full_name character varying (255)	debt_on_2024_03_31 numeric
1	Иванов Иван Иванович	2024.13

Рисунок 36 – Результат выполнения функции get_client_debt

7.4. Разработка процедуры

Задача: Создать процедуру для закрытия начислений за определённый месяц путём автоматического распределения платежей клиента по принципу FIFO (первым пришёл — первым ушёл).

Скрипт создания процедуры представлен на рисунке 37.

```

1 CREATE OR REPLACE PROCEDURE allocate_payments_fifo(
2     p_client_id INTEGER,
3     p_target_period DATE
4 )
5 LANGUAGE plpgsql
6 AS $$
7 DECLARE
8     v_payment_record RECORD;
9     v_accrual_record RECORD;
10    v_remaining_amount DECIMAL(15,2);
11    v_allocated DECIMAL(15,2);
12 BEGIN
13     CREATE TEMP TABLE IF NOT EXISTS temp_payments AS
14     SELECT
15         p.payment_id,
16         p.amount - COALESCE(SUM(pa.allocated_amount), 0) AS remaining
17     FROM payments p
18     LEFT JOIN payment_allocations pa ON p.payment_id = pa.payment_id
19     WHERE p.client_id = p_client_id
20     GROUP BY p.payment_id, p.amount
21     HAVING p.amount - COALESCE(SUM(pa.allocated_amount), 0) > 0;
22
23     FOR v_accrual_record IN
24     SELECT
25         a.accrual_id,
26         a.amount - COALESCE(SUM(pa.allocated_amount), 0) AS debt
27     FROM contracts c
28     JOIN accruals a ON c.contract_id = a.contract_id
29     LEFT JOIN payment_allocations pa ON a.accrual_id = pa.accrual_id
30     WHERE c.client_id = p_client_id
31           AND a.period <= p_target_period
32           AND a.status != 'Оплачено'
33     GROUP BY a.accrual_id, a.amount
34     HAVING a.amount - COALESCE(SUM(pa.allocated_amount), 0) > 0
35     ORDER BY a.period, a.created_date
36     LOOP
37         FOR v_payment_record IN
38         SELECT * FROM temp_payments WHERE remaining > 0 ORDER BY payment_id
39         LOOP
40             IF v_payment_record.remaining >= v_accrual_record.debt THEN
41                 v_allocated := v_accrual_record.debt;
42                 UPDATE temp_payments
43                 SET remaining = remaining - v_allocated
44                 WHERE payment_id = v_payment_record.payment_id;
45             ELSE
46                 v_allocated := v_payment_record.remaining;
47                 UPDATE temp_payments
48                 SET remaining = 0
49                 WHERE payment_id = v_payment_record.payment_id;
50             END IF;
51
52             INSERT INTO payment_allocations (payment_id, accrual_id, allocated_amount)
53             VALUES (v_payment_record.payment_id, v_accrual_record.accrual_id, v_allocated);
54
55             v_accrual_record.debt := v_accrual_record.debt - v_allocated;
56
57             EXIT WHEN v_accrual_record.debt <= 0;
58         END LOOP;
59
60         IF v_accrual_record.debt <= 0 THEN
61             UPDATE accruals SET status = 'Оплачено' WHERE accrual_id = v_accrual_record.accrual_id;
62         ELSE
63             UPDATE accruals SET status = 'Частично оплачено' WHERE accrual_id = v_accrual_record.accrual_id;
64         END IF;
65     END LOOP;
66
67     DROP TABLE temp_payments;
68 END;
69 $$;

```

Рисунок 37. – Создание процедуры

Распределение платежей до и после вызова процедуры представлено на рисунке 38.

	full_name character varying (255)	accrual_id integer	period date	amount numeric (15,2)	status character varying (20)	paid numeric
1	Иванов Иван Иванович	1	2024-01-...	568.75	Оплачено	568.75
2	Иванов Иван Иванович	4	2024-01-...	615.00	Оплачено	615.00
3	Иванов Иван Иванович	5	2024-02-...	615.00	Оплачено	615.00
4	Иванов Иван Иванович	2	2024-02-...	277.88	Частично оплачено	277.88
5	Иванов Иван Иванович	3	2024-03-...	321.75	Оплачено	321.75
6	Иванов Иван Иванович	6	2024-03-...	615.00	Частично оплачено	98.75
7	Иванов Иван Иванович	25	2024-04-...	0.00	Оплачено	[null]
8	Иванов Иван Иванович	26	2024-04-...	0.00	Оплачено	[null]
	full_name character varying (255)	accrual_id integer	period date	amount numeric (15,2)	status character varying (20)	paid numeric
1	Иванов Иван Иванович	1	2024-01-...	568.75	Оплачено	568.75
2	Иванов Иван Иванович	4	2024-01-...	615.00	Оплачено	615.00
3	Иванов Иван Иванович	5	2024-02-...	615.00	Оплачено	615.00
4	Иванов Иван Иванович	2	2024-02-...	277.88	Частично оплачено	277.88
5	Иванов Иван Иванович	3	2024-03-...	321.75	Оплачено	321.75
6	Иванов Иван Иванович	6	2024-03-...	615.00	Частично оплачено	598.75
7	Иванов Иван Иванович	25	2024-04-...	0.00	Оплачено	[null]
8	Иванов Иван Иванович	26	2024-04-...	0.00	Оплачено	[null]

Рисунок 38. – Распределение платежей до и после вызова процедуры

7.5. Разработка триггеров

Триггер 1: Автоматическое обновление статуса начисления при полной оплате

Задача: Создать триггер, который после вставки или обновления в таблице payment_allocations проверяет, полностью ли оплачено начисление, и автоматически меняет его статус на «Оплачено».

Скрипт для создания триггера представлен на рисунке 39.

```

1 DROP TRIGGER IF EXISTS trg_update_accrual_status ON payment_allocations;
2 DROP FUNCTION IF EXISTS update_accrual_status();
3
4 CREATE OR REPLACE FUNCTION update_accrual_status()
5 RETURNS TRIGGER
6 LANGUAGE plpgsql
7 AS $$
8 BEGIN
9     IF (
10         SELECT a.amount <= COALESCE(SUM(pa.allocated_amount), 0)
11         FROM accruals a
12         LEFT JOIN payment_allocations pa ON a.accrual_id = pa.accrual_id
13         WHERE a.accrual_id = COALESCE(NEW.accrual_id, OLD.accrual_id)
14         GROUP BY a.accrual_id, a.amount
15     ) THEN
16         UPDATE accruals
17         SET status = 'Оплачено'
18         WHERE accrual_id = COALESCE(NEW.accrual_id, OLD.accrual_id);
19     ELSE
20         UPDATE accruals
21         SET status = 'Частично оплачено'
22         WHERE accrual_id = COALESCE(NEW.accrual_id, OLD.accrual_id)
23         AND status != 'Оплачено';
24     END IF;
25     RETURN NEW;
26 END;
27 $$;
28
29 CREATE TRIGGER trg_update_accrual_status
30 AFTER INSERT OR UPDATE OF allocated_amount OR DELETE
31 ON payment_allocations
32 FOR EACH ROW
33 EXECUTE FUNCTION update_accrual_status();

```

Рисунок 39. – Скрипт для создания триггера

Триггер 2: Проверка даты показания при вставке

Задача: Создать триггер, который при вставке нового показания проверяет, чтобы дата нового показания (`reading_date`) была позже даты последнего существующего показания для данного счётчика. Это предотвратит хронологические ошибки.

Скрипт для создания триггера представлен на рисунке 40.

```

1  DROP TRIGGER IF EXISTS trg_check_reading_date ON meter_readings;
2  DROP FUNCTION IF EXISTS check_reading_date_sequence();
3
4  CREATE OR REPLACE FUNCTION check_reading_date_sequence()
5  RETURNS TRIGGER
6  LANGUAGE plpgsql
7  AS $$
8  DECLARE
9      last_reading_date DATE;
10 BEGIN
11     SELECT MAX(reading_date) INTO last_reading_date
12     FROM meter_readings
13     WHERE meter_id = NEW.meter_id;
14
15     IF last_reading_date IS NOT NULL AND NEW.reading_date <= last_reading_date THEN
16         RAISE EXCEPTION 'Дата нового показания (%) должна быть позже последней существующей даты (%) для счётчика %',
17             NEW.reading_date, last_reading_date, NEW.meter_id;
18     END IF;
19
20     RETURN NEW;
21 END;
22 $$;
23
24 CREATE TRIGGER trg_check_reading_date
25 BEFORE INSERT
26 ON meter_readings
27 FOR EACH ROW
28 EXECUTE FUNCTION check_reading_date_sequence();

```

Рисунок 40. – Скрипт для создания триггера

ЗАКЛЮЧЕНИЕ

В ходе выполнения расчётно-графической работы по дисциплине «Системы управления базами данных» была успешно спроектирована и реализована база данных для автоматизации учёта оплаты коммунальных услуг (водоснабжение, электроснабжение, газоснабжение), предоставляемых физическим и юридическим лицам.

В процессе работы были последовательно решены все поставленные задачи:

- Анализ предметной области позволил выделить ключевые сущности, атрибуты и бизнес-правила, определяющие логику функционирования системы учёта.
- Концептуальное и логическое проектирование обеспечило создание непротиворечивой модели данных, отражающей взаимосвязи между клиентами, договорами, услугами, приборами учёта, показаниями, тарифами, начислениями и платежами.
- Физическая реализация в СУБД PostgreSQL включала создание девяти взаимосвязанных таблиц, определение типов данных, установку ограничений целостности (PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE) и создание индексов для оптимизации производительности.
- Разработка разнообразных SQL-запросов продемонстрировала возможности извлечения и анализа данных: от простой выборки до сложных запросов с оконными функциями, подзапросами, агрегацией и использованием материализованных представлений.
- Создание программных объектов БД (курсоры, хранимые функции, процедуры, триггеры) позволило реализовать бизнес-логику непосредственно на уровне базы данных, обеспечив автоматизацию расчётов, распределения платежей, контроля хронологии показаний и обновления статусов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Малков, О. Б. Работа с СУБД PostgreSQL : учеб. пособие / О. Б. Малков, М. П. Маркова, М. В. Девятерикова ; Минобрнауки России, Ом. гос. техн. ун-т. – Омск : Изд-во ОмГТУ, 2023.
2. Моргунов, Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил.